

Plateforme de Gestion des Contrats de Maintenance

SolidWall Consulting

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de

Licence Appliquée en Informatique

Réalisé par :

Rihab CHOUROU

Eya GHARBI

Encadré par :

Encadrant Académique : [Nom]

Encadrant Professionnel : [Nom]

Année Universitaire 2025-2026

Table des matières

Introduction générale	1
Webographie	2
Cadre général du projet	2
I Présentation de l'organisme d'accueil	4
1 Services de SolidWall Consulting	4
2 Organigramme de l'organisme	5
II Cadre du projet	5
1 Contexte du projet	5
2 Analyse de l'existant	5
2.1 Étude de l'existant	5
2.2 Critique de l'existant	9
2.3 Tableau comparative des solutions	10
2.4 Solution proposée	10
III Méthodologie adaptée	11
1 Méthodologie classique	11
1.1 Principes de la méthodologie classique	12
2 Méthodologie Agile	12
2.1 Les valeurs agiles	12
3 Choix de la méthode de travail	13
3.1 Choix de l'approche Agile	13
3.2 Les principales méthodes Agiles	13
3.3 Choix de la méthode Scrum	14
4 Fondements de SCRUM	14
5 Rôles de SCRUM	15
6 Evènements Scrum	15
7 Artéfacts Scrum	16
8 Cycle de vie	16
IV Langage de modélisation UML – Unified Modeling Language	16

Préparation du projet	18
I	Spécification des besoins 20
1	Identification des acteurs 20
2	Identification des besoins fonctionnels 21
2.1	Authentification et Gestion de Compte 21
2.2	Gestion des Utilisateurs et Demandes d'Inscription 21
2.3	Système de Rôles et Permissions 22
2.4	Gestion des Clients et Projets 22
2.5	Gestion des Contrats de Maintenance 23
2.6	Gestion des Demandes d'Intervention 23
2.7	Gestion Avancée des Tickets de Maintenance 24
2.8	Tableaux de Bord, Traçabilité et Documentation 24
3	Identification des besoins non fonctionnels 25
II	Modélisation des besoins 26
III	Pilotage du projet avec Scrum 28
1	Rôles de SCRUM 28
1.1	Backlog de produit 28
2	Planification des sprints 48
2.1	Sprint 1 - Gestion des Utilisateurs, Clients et Authentification . . . 49
2.2	Sprint 2 - Gestion des Projets et Contrats 49
2.3	Sprint 3 - Gestion des Demandes et Tickets 50
2.4	Sprint 4 - Collaboration, Documents et IA 50
3	Planning prévisionnel (Diagramme de Gantt) 51
IV	Environnement de Développement 52
1	Environnement matériel 52
2	Environnement logiciel 52
2.1	Langage de programmation et framework 52
2.2	Outils de développement et modélisation 53
2.3	Systèmes de Gestion de Bases de données 53
2.4	Outil de Gestion de Projet Scrum 54
2.5	Environnement de composition de documents 54
2.6	Outil de design graphique 54
2.7	Outils de test et qualité logicielle 54
V	Architecture de l'application 55

1	Architecture physique	55
2	Architecture logique	55
VI	Conclusion	56
Gestion des Utilisateurs et Authentification		57
I	Sprint Planning Meeting	59
1	Objectif du Sprint	59
1.1	Backlog du premier sprint	59
II	Analyse	65
1	Diagramme des cas d'utilisation du premier sprint	66
2	Description textuelle de cas d'utilisation « Changer mot de passe »	67
3	Description textuelle de cas d'utilisation « Demander accès via OAuth »	68
4	Analyse de la fonctionnalité « Gérer utilisateurs »	68
4.1	Raffinement de cas d'utilisation « Gérer utilisateurs »	68
4.2	Description textuelle de cas d'utilisation « Consulter liste utilisateurs »	69
4.3	Description textuelle de cas d'utilisation « Ajouter utilisateur »	70
4.4	Description textuelle de cas d'utilisation « Modifier utilisateur »	70
4.5	Description textuelle de cas d'utilisation « Désactiver un utilisateur »	71
5	Analyse de la fonctionnalité « Gérer profil »	71
5.1	Raffinement de cas d'utilisation « Gérer Profil »	71
5.2	Description textuelle de cas d'utilisation « Consulter Profil »	72
5.3	Description textuelle de cas d'utilisation « Modifier Profil »	72
6	Analyse de la fonctionnalité « Gérer rôles »	72
6.1	Raffinement de cas d'utilisation « Gérer rôles »	72
6.2	Description textuelle de cas d'utilisation « Consulter liste rôles »	73
6.3	Description textuelle de cas d'utilisation « Ajouter rôle »	74
6.4	Description textuelle de cas d'utilisation « Modifier rôle »	74
6.5	Description textuelle de cas d'utilisation « Archiver rôle »	75
6.6	Description textuelle de cas d'utilisation « Supprimer rôle »	75
6.7	Description textuelle de cas d'utilisation « Attribuer rôle à utilisateur »	76
6.8	Description textuelle de cas d'utilisation « Retirer rôle d'un utilisateur »	76
7	Description des cas d'utilisation « Gérer permissions »	76
7.1	Raffinement de cas d'utilisation « Gérer permissions »	76

7.2	Description textuelle de cas d'utilisation « Consulter liste des permissions »	77
7.3	Description textuelle de cas d'utilisation « Attribuer permission à rôle »	78
7.4	Description textuelle de cas d'utilisation « Retirer permission d'un rôle »	78
8	Description des cas d'utilisation « Gérer clients »	78
8.1	Raffinement de cas d'utilisation « Gérer clients »	78
8.2	Description textuelle de cas d'utilisation « Consulter liste clients »	79
8.3	Description textuelle de cas d'utilisation « Ajouter client »	80
8.4	Description textuelle de cas d'utilisation « Modifier client »	81
8.5	Description textuelle de cas d'utilisation « Consulter profil client »	81
8.6	Description textuelle de cas d'utilisation « Désactiver client » . . .	82
8.7	Description textuelle de cas d'utilisation « Réactiver client » . . .	82
8.8	Description textuelle de cas d'utilisation « Gérer notes privées client »	83
III	Diagramme de classes de première sprint	84
IV	Schéma relationnel	84
V	Réalisation	85
1	Diagramme de compositions	85
2	Description des interfaces	85
VI	Tests et Validation	86
1	Stratégie de tests adoptée	86
1.1	Tests unitaires	86
1.2	Tests d'intégration manuels de l'API	87
1.3	Tests End-to-End	87
	Réalisation	87
I	Analyse	93
1	Diagramme des cas d'utilisation du Sprint 2	93
2	Description textuelle des cas d'utilisation	93
II	Conception	94
1	Diagrammes de séquence	94
2	Modèle de données	94
III	Réalisation	95

	1	Architecture technique	95
	2	Interfaces utilisateur	95
IV	Tests		96
	1	Tests unitaires	96
	2	Tests d'intégration	96
Chapitre 5			96
I	Analyse		102
	1	Diagramme des cas d'utilisation du Sprint 3	102
	2	Description textuelle des cas d'utilisation	102
II	Conception		103
	1	Diagrammes de séquence	103
	2	Modèle de données	103
III	Réalisation		104
	1	Architecture technique	104
	2	Interfaces utilisateur	104
IV	Tests		105
	1	Tests unitaires	105
	2	Tests d'intégration	105
Chapitre 6			105
I	Analyse		113
	1	Diagramme des cas d'utilisation du Sprint 4	113
	2	Description textuelle des cas d'utilisation	113
II	Conception		114
	1	Diagrammes de séquence	114
	2	Modèle de données	114
III	Réalisation		115
	1	Architecture technique	115
	2	Interfaces utilisateur	115
IV	Tests		116
	1	Tests unitaires	116
	2	Tests d'intégration	116

Introduction générale

Dans un environnement technologique en constante évolution, la maintenance des systèmes informatiques représente un enjeu stratégique majeur pour les entreprises de services numériques.

Les organisations doivent concilier la réactivité face aux incidents, le respect des accords de niveau de service (SLA) et une traçabilité des interventions.

Au sein de **SolidWall Consulting**, société spécialisée dans le développement web et mobile, la gestion actuelle des contrats de maintenance repose sur des processus manuels fragmentés : documents Word dispersés, échanges par courriels et affectation manuelle des tickets sans traçabilité. Cette approche génère des risques opérationnels significatifs et limite la capacité à respecter les engagements de service.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, visant à concevoir et réaliser **SolidMaint**, une plateforme web intégrée de gestion des contrats de maintenance. L'objectif principal consiste à automatiser et centraliser l'ensemble du processus métier, depuis la soumission des demandes clients jusqu'à la résolution des incidents, tout en garantissant une traçabilité complète et le respect des engagements contractuels.

Ce rapport s'articule autour de six chapitres retraçant la démarche complète du proje :

Le **premier chapitre** établit le cadre général avec l'analyse de l'existant, l'étude comparative des solutions du marché, et la définition de la solution proposée.

Le **deuxième chapitre** détaille la spécification des besoins fonctionnels et non fonctionnels, l'identification des acteurs du système, la modélisation UML des cas d'utilisation, la planification Scrum avec constitution du Product Backlog, l'environnement de développement, et l'architecture technique de l'application.

Le **troisième chapitre** présente la réalisation du premier sprint : authentification sécurisée, système rôles et permissions, gestion des utilisateurs et des clients.

Les **chapitres suivants** couvrent les sprints restants : gestion des contrats, processus des demandes et tickets, fonctionnalités de collaboration et intelligence artificielle, accompagnés des tests et validations.

Webographie

1. **Atlassian**, Qu'est-ce que la méthodologie Agile? - <https://www.atlassian.com/fr/agile>
2. **Jira Software**, Guide utilisateur - <https://www.atlassian.com/software/jira>
3. **ManageEngine**, Documentation officielle - <https://www.manageengine.com>
4. **NestJS**, Documentation - <https://docs.nestjs.com>
5. **Next.js**, Documentation officielle - <https://nextjs.org/docs>
6. **OMG**, Why is UML important? - <https://www.omg.org/uml/why-uml-is-important.htm>
7. **React**, Guide officiel - <https://react.dev>
8. **SolidWall Consulting**, Site officiel - <https://solidwall.com.tn/>



CHAPITRE 1

Cadre général du projet



Sommaire

1

I	Présentation de l'organisme d'accueil	4
II	Cadre du projet	5
III	Méthodologie adaptée	11
IV	Langage de modélisation UML – Uni- fied Modeling Language	16

Introduction

Ce chapitre pose les fondations du projet en présentant le contexte organisationnel et technique de sa réalisation. Après la présentation de **SolidWall Consulting**, entreprise d'accueil du stage, nous analysons les processus actuels de gestion de maintenance et leurs limitations. L'étude comparative des solutions du marché permet d'identifier les axes d'amélioration et de définir les contours de **SolidMaint**. Enfin, nous exposons la méthodologie Scrum, le langage UML et la planification prévisionnelle du développement.

I Présentation de l'organisme d'accueil

«Solid Wall Consulting est une société de services informatiques, spécialisée dans le développement web, le développement mobile et l'intégration de solutions et d'infrastructures systèmes « Microsoft ». Forte d'une équipe dynamique et diversifiée et de profils reconnus, l'entreprise a pour objectif constant d'apporter une forte valeur ajoutée et de proposer des solutions modernes permettant aux entreprises de se démarquer sur leurs marchés respectifs. » [1] Fondée avec la vision d'accompagner les organisations publiques et privées vers l'excellence technologique et opérationnelle, Solid Wall Consulting s'est rapidement imposée comme un partenaire de confiance à l'international. » [2].



FIGURE I.1 – Logo SolidWall Consulting

1 Services de SolidWall Consulting

SolidWall Consulting propose des services conseil pour accompagner ses clients dans la réalisation de leurs projets et assure la qualité de ses prestations ainsi que le développement de solutions numériques.

L'approche agile de **SolidWall Consulting** garantit une gestion optimisée et par conséquent, d'un résultat de qualité.

2 Organigramme de l'organisme

Pour mettre en valeur la répartition des responsabilités et des fonctions au sein de l'organisation, nous présentons la structure hiérarchique de SolidWall Consulting avec ses principales divisions ainsi que les relations de subordination entre les différentes unités et postes, comme le montre la figure suivante.

II Cadre du projet

1 Contexte du projet

Dans le cadre de notre projet de fin d'études à l'**Institut Supérieur des Études Technologiques de Radès**, nous avons effectué un stage de quatre mois au sein de **SolidWall Consulting**, du 02 février au 31 mai 2026. Notre mission consiste à concevoir et à réaliser « **SolidMaint : une application web de gestion des contrats de maintenance** ».

Cette solution vise à automatiser et à centraliser l'ensemble du processus métier, garantissant ainsi une traçabilité complète des interventions techniques ainsi que le respect rigoureux des engagements contractuels.

2 Analyse de l'existant

Pour mieux comprendre les besoins, nous allons étudier les solutions existantes à deux niveaux : celles déjà en place au sein de l'entreprise **SolidWall Consulting**, puis celles utilisées dans le monde.

2.1 Étude de l'existant

L'étude de l'existant constitue une étape fondamentale de notre démarche projet. Elle permet d'identifier les processus actuels, d'analyser leurs limitations et de définir les axes d'amélioration nécessaires pour concevoir une solution parfaitement adaptée aux besoins de SolidWall Consulting.

a) Étude de l'existant au niveau de l'entreprise

– Processus de gestion des contrats de maintenance

L'entreprise utilise actuellement des documents Word pour stocker et suivre les informations relatives aux contrats. Ces documents permettent aux responsables de formaliser la relation contractuelle avec les clients et de définir les engagements de chaque partie concernant les services de maintenance.

modifications sont effectuées directement sur les documents existants.

b) Étude de l'existant à l'échelle internationale

Avant de concevoir notre propre solution, nous allons étudier les solutions disponibles sur le marché afin d'analyser les fonctionnalités proposées. Cette démarche permet d'identifier les bonnes pratiques et d'orienter efficacement le développement de notre système.

Dans ce cadre, nous avons étudié plusieurs solutions existantes couvrant différents segments du marché : solutions open-source, solutions SaaS et solutions entreprise.

– GLPI (Gestion Libre de Parc Informatique)

Cette solution de gestion des services informatiques, développée en 2003 et maintenue par la société française Teclib, est open source, ce qui signifie que son code est librement accessible et peut être modifié et adapté selon les besoins des utilisateurs. Principalement utilisée par les équipes internes d'une organisation, elle peut également, selon sa configuration, recevoir des demandes externes. Elle assure un système de gestion de tickets structurant les demandes des utilisateurs. De plus, elle prend en charge différents canaux de communication, garantissant ainsi la centralisation des échanges. La version communautaire est gratuite, mais nécessite une formation de 2 à 3 jours pour les administrateurs. La version entreprise, avec support professionnel, coûte à partir de **1500€** par an.

ID	TITLE	ENTITY	PRIORITY	STATUS	REQUESTER - REQUESTED	ASSIGNED TO - TECHNICIAN	OPENING DATE	LAST UPDATE *	TIME TO RESOLVE - PROGRESS
11769	facilitate compelling users	Root • Guikowasi Runufubatsi • Kiba LLC	High	Pending	Laron Champlin	Wava Brekke	09-09-2017 17:45	22-12-2021 16:37	
4647	synthesize next-generation schemas	Root • Guikowasi Runufubatsi • Spenn, Hain and Marack	Low	New	Calle Farrell	Jaylon Hartmann	11-08-2015 15:26	14-12-2021 21:12	
13429	facilitate strategic partnerships	Root • Murray and Soles • Zach, Kerman and Cummings	Low	Processing (assigned)	Viviane Rogahn		15-03-2018 16:30	08-12-2021 16:24	
17459	transform customized synergies	Root • Cronin, Turcotte and Cummings • Schaefer, O'Connor and Lattaack	Low	Pending	Felton Reichert Anjali Abshire Maurine Yundt		11-07-2019 16:48	06-12-2021 14:59	
17460	strategize customized applications	Root • Cronin, Turcotte and Cummings • Schaefer, O'Connor and Lattaack	Low	Pending	Anjali Abshire Maurine Yundt	William McDermott	11-07-2019 17:24	06-12-2021 14:59	
17905	optimize cross-platform portals	Root • Cronin, Turcotte and Cummings • Schaefer, O'Connor and Lattaack	Low	Pending	Maurine Yundt	William McDermott	16-09-2019 18:00	06-12-2021 14:59	
18425	repurpose granular mindshare	Root • Adams, Ortiz and Anderson	Medium	Processing (assigned)	Manley Boyer		08-11-2019 16:25	06-12-2021 14:59	
18427	optimize cross-media e-commerce	Root • Adams, Ortiz and Anderson	Medium	Processing (assigned)	Manley Boyer		08-11-2019 17:10	06-12-2021 14:59	
18605	integrate strategic schemas	Root • Skiles, Cronin and Zboncak • Hartmann, White and Pfeiffer	Medium	Pending	Lisette Strouh		29-11-2019 09:52	06-12-2021 14:59	

FIGURE I.5 – Interface de GLPI

– Freshdesk

Cette solution cloud de service desk, développée en 2011 par la société indienne Freshworks, est une solution SaaS hébergée dans le cloud et accessible via navigateur web. **Freshdesk**

se caractérise par sa facilité de déploiement et une maintenance entièrement assurée par le fournisseur.

Elle propose une interface intuitive pour la gestion des tickets avec support multicanal (courriel, chat, téléphone, réseaux sociaux). **Freshdesk** intègre des fonctionnalités d'automatisation avancées et d'intelligence artificielle pour la catégorisation automatique des tickets.

La solution propose un plan gratuit limité, tandis que les offres payantes démarrent à partir de **15€ par agent et par mois**

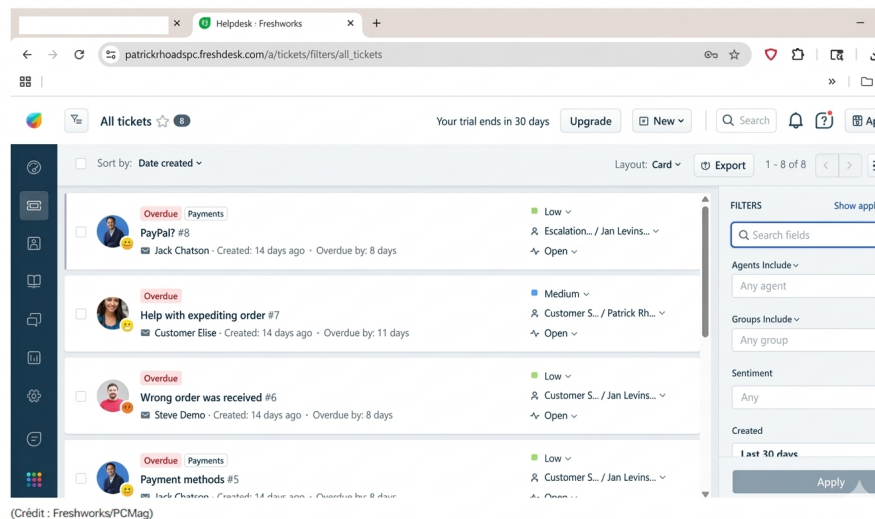
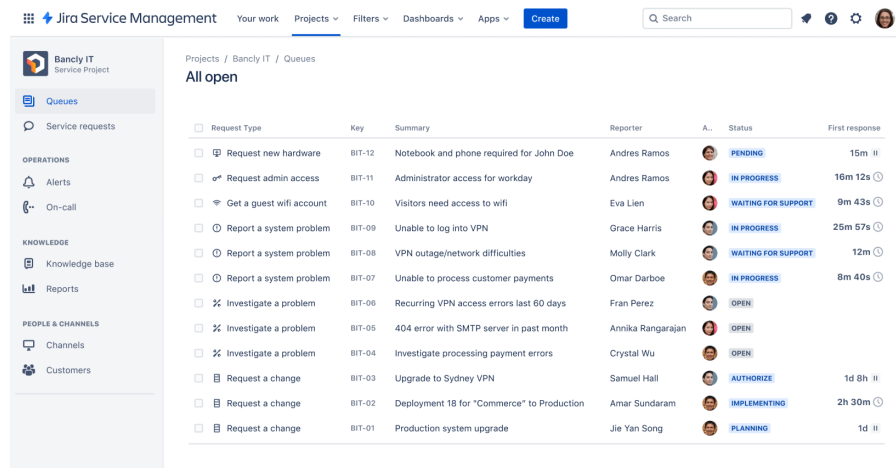


FIGURE I.6 – Interface de Freshdesk

– Jira Service Management (Atlassian)

Cette solution, développée en 2002 et maintenue par Atlassian, une entreprise australienne leader dans les outils de gestion de projet, cette solution entreprise s'intègre parfaitement à l'écosystème Atlassian (Jira Software, Confluence). Elle propose des fonctionnalités avancées de gestion des incidents, des changements et des actifs, ainsi que des capacités d'IA pour l'automatisation et la prédiction. Les tarifs démarrent à **20€/agent/mois**, mais augmentent considérablement avec les modules avancés. La maîtrise de l'écosystème Atlassian est nécessaire pour exploiter pleinement ses capacités.



The screenshot displays the Jira Service Management interface for the 'Bancly IT' project. The left sidebar contains navigation options like 'Queues', 'Service requests', 'OPERATIONS', 'Alerts', 'On-call', 'KNOWLEDGE', 'Reports', 'PEOPLE & CHANNELS', and 'Customers'. The main area shows a table of 'All open' requests.

Request Type	Key	Summary	Reporter	Status	First response
Request new hardware	BIT-12	Notebook and phone required for John Doe	Andres Ramos	PENDING	15m
Request admin access	BIT-11	Administrator access for workday	Andres Ramos	IN PROGRESS	16m 12s
Get a guest wifi account	BIT-10	Visitors need access to wifi	Eva Lien	WAITING FOR SUPPORT	9m 43s
Report a system problem	BIT-09	Unable to log into VPN	Grace Harris	IN PROGRESS	25m 57s
Report a system problem	BIT-08	VPN outage/network difficulties	Molly Clark	WAITING FOR SUPPORT	12m
Report a system problem	BIT-07	Unable to process customer payments	Omar Darboe	IN PROGRESS	8m 40s
Investigate a problem	BIT-06	Recurring VPN access errors last 60 days	Fran Perez	OPEN	
Investigate a problem	BIT-05	404 error with SMTP server in past month	Annika Rangarajan	OPEN	
Investigate a problem	BIT-04	Investigate processing payment errors	Crystal Wu	OPEN	
Request a change	BIT-03	Upgrade to Sydney VPN	Samuel Hall	AUTHORIZE	1d 8h
Request a change	BIT-02	Deployment 18 for "Commerce" to Production	Amar Sundaram	IMPLEMENTING	2h 30m
Request a change	BIT-01	Production system upgrade	Jie Yan Song	PLANNING	1d

FIGURE I.7 – Interface de Jira Service Management

2.2 Critique de l'existant

Actuellement, la gestion des contrats de maintenance au sein de la direction de gestion SolidMaint se fait par échange de courriels. Cette approche présente certaines limites.

Limitations du processus actuel

- **processus fragmenté** : La communication via e-mail fragmente la gestion. Les informations étant dispersées, le suivi des composants manquants devient complexe. En l'absence de structure centralisée, il devient difficile de retrouver l'historique complet d'une intervention.
- **Absence de traçabilité** : L'absence de système de traçabilité rend le suivi des interventions difficile. Cela peut entraîner une perte de visibilité sur leur état, en cas de conflit avec un client ou lors d'un audit interne, ce qui représente un risque pour la crédibilité et la fiabilité de la société.
- **Risque de perte d'informations** : En raison de la fragmentation des processus, il existe un risque accru de perte d'informations importantes. Les documents sont stockés de manière non structurée, ce qui rend leur suivi et leur consolidation difficiles.
- **Complexité de la communication** : la communication au sein de l'équipe et avec les clients repose sur des échanges dispersés (courriels, messages), ce qui rend le flux des demandes peu clair et entraîne des retards dans le traitement des interventions.

Limitations des solutions existantes

Complexité d'utilisation et courbe d'apprentissage

Les solutions existantes présentent une complexité d'usage importante. GLPI nécessite une formation de 2 à 3 jours pour les administrateurs avec une interface peu intuitive. Jira Service Management requiert une connaissance approfondie de l'écosystème Atlassian, créant une dépendance technique majeure.

Coût des licences et investissement financier

L'investissement financier représente un obstacle majeur. Les tarifs s'échelonnent de 15€/agent/mois pour Freshdesk à 24€/utilisateur/mois pour Helpdesk, tandis que Jira Service Management coûte 20€/agent/mois. GLPI, bien que gratuit en version communautaire, nécessite 1500€/an minimum pour le support entreprise. Pour une équipe de 10 utilisateurs, le coût annuel varie de 1500€ à 12000€.

Dépendance aux extensions et problèmes d'intégration

Les solutions du marché souffrent d'une architecture modulaire générant des coûts cachés. De nombreuses fonctionnalités avancées nécessitent des extensions payantes supplémentaires. Les mises à jour engendrent des problèmes de compatibilité, et l'intégration avec des systèmes existants nécessite souvent des développements spécifiques onéreux.

2.3 Tableau comparative des solutions

Ces limitations se reflètent clairement dans l'analyse comparative des principales solutions du marché. Le tableau suivant synthétise les forces et faiblesses de chaque solution selon des critères fonctionnels, techniques et financiers :

Critères	GLPI	Jira SM	Freshdesk
Gestion tickets	✓	✓	✓
Gestion contrats	Limité	Limité	×
Messagerie intégrée	×	Limité	✓
IA/Suggestions	×	✓	✓
Facilité d'utilisation	Moyenne	Moyenne	Facile
Coût (10 users/an)	1500€	2000€	1800€

TABLE I.1 – Tableau comparatif des solutions de gestion de maintenance existantes

Note sur les coûts : Les tarifs indiqués correspondent au coût total annuel pour une équipe de 10 utilisateurs, calculés selon les grilles tarifaires officielles de chaque éditeur. Ces montants incluent uniquement les fonctionnalités de base et peuvent augmenter significativement avec l'ajout de modules complémentaires, de support premium ou de services de personnalisation.

2.4 Solution proposée

Dans le contexte des défis exposés, nous proposons la mise en place de **SolidMaint**, une solution web intégrée pour optimiser et accélérer le processus de gestion des contrats de maintenance.

SolidMaint s'articule autour de quatre modules fonctionnels complémentaires :

Module 1 : Authentification, Gestion des Utilisateurs et Clients

Ce module constitue le socle sécuritaire de la plateforme avec un système d'authentification robuste et un contrôle d'accès basé sur les rôles (RBAC). Il assure la gestion des profils utilisateurs, des clients et l'historique des activités pour garantir la traçabilité.

Module 2 : Gestion des Projets et Contrats

Ce module centralise la gestion des projets et des contrats de maintenance par client. Il garantit la traçabilité complète et optimise le respect des engagements contractuels grâce aux alertes préventives.

Module 3 : Gestion des Demandes et Tickets

Cœur métier de la plateforme, ce module structure le cycle complet des demandes clients depuis leur soumission jusqu'à leur clôture. Il génère automatiquement des tickets, permet leur affectation aux développeurs et assure le suivi du temps passé.

Module 4 : Collaboration et Intelligence Artificielle

Ce module regroupe les fonctionnalités avancées : messagerie temps réel, gestion documentaire et notifications multicanaux. Les fonctionnalités d'IA assurent la catégorisation automatique, la priorisation intelligente et la recommandation de solutions basées sur l'historique.

SolidMaint apportera une réponse concrète aux défis identifiés en matière de gestion des contrats, d'interventions et de communication client. Grâce à l'intégration de l'intelligence artificielle et d'une interface intuitive, cette plateforme centralisera tous les processus de maintenance dans un environnement unifié.

III Méthodologie adaptée

Le choix méthodologique est déterminant pour la réussite d'un projet de développement. Une méthodologie appropriée structure le travail, optimise la gestion des ressources et garantit la qualité du livrable dans le respect des délais impartis.

1 Méthodologie classique

La méthodologie classique de la gestion de projet s'articule autour d'une série d'étapes consécutives, où l'approbation de chaque phase est essentielle pour progresser vers la suivante. Les projets se déroulent selon des phases distinctes et structurées, représentées par différents modèles de cycle de vie : cascade, V ou spirale.

Cette approche convient particulièrement aux projets dont les besoins sont stables et exhaustivement

définis dès le départ. Elle offre une planification détaillée facilitant le suivi des délais et des coûts. Cependant, sa rigidité constitue une limitation majeure face aux changements de besoins en cours de développement, rendant difficile l'intégration de retours clients durant le processus.

Afin de mieux comprendre le fonctionnement de cette approche, il convient de présenter ses caractéristiques fondamentales.

1.1 Principes de la méthodologie classique

Cette approche repose sur une organisation linéaire où chaque phase doit être validée avant progression. Elle exige une définition exhaustive des besoins en phase initiale et une planification servant de base à la gestion. Le contrôle et la validation s'effectuent de manière progressive tout au long du cycle de vie.

Dans ce contexte, la méthodologie Agile a été adoptée comme une alternative à cette approche.

2 Méthodologie Agile

« La méthodologie Agile décompose le travail en phases, mettant l'accent sur la livraison continue et l'amélioration. Agile profite aux équipes en permettant une planification adaptative, une exécution rapide et une évaluation continue, garantissant des résultats plus réactifs et plus réussis. » [3]

La méthodologie Agile est adoptée pour répondre rapidement aux retours des clients. Elle valorise la collaboration plutôt que la négociation contractuelle, en proposant d'abord une version minimale, puis en intégrant des fonctionnalités supplémentaires. L'approche Agile est une démarche moderne qui offre souplesse et performance dans la gestion de projet.

Afin de mieux comprendre les fondements de cette approche, il est important de présenter ses valeurs essentielles.

2.1 Les valeurs agiles

Les valeurs de la méthodologie Agile constituent des principes fondamentaux qui guident l'organisation et le fonctionnement des projets agiles. Elles permettent d'améliorer la collaboration, la qualité du travail et la réactivité de l'équipe face aux changements.

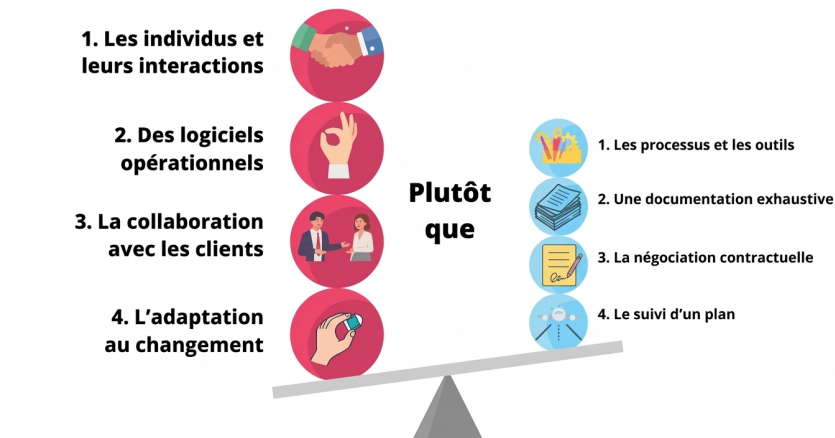


FIGURE I.8 – les valeurs agiles

Cette figure présente les quatre valeurs principales de la méthode Agile qui mettent l'accent sur :

- Les individus et leurs interactions plus que les processus et les outils
- Des logiciels opérationnels plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

À partir de ces valeurs, plusieurs méthodes Agile ont été développées et peuvent être comparées selon différents critères.

3 Choix de la méthode de travail

Après avoir présenté les différentes approches et méthodes de gestion de projet, il convient de justifier les choix adoptés dans le cadre de ce travail.

3.1 Choix de l'approche Agile

Le domaine de la maintenance impose des besoins variables et imprévisibles, rendant impossible une planification exhaustive. Cette réalité justifie l'adoption d'une méthodologie agile privilégiant la flexibilité et la collaboration continue avec les clients. Parmi les méthodes disponibles, Scrum offre le cadre itératif le plus adapté à notre contexte.

3.2 Les principales méthodes Agiles

Le tableau suivant met en évidence les méthodes Agile les plus couramment utilisées, selon différents critères de comparaison.

Critères	Scrum	Kanban	Extreme Programming (XP)
Type de projet	Incrémental	Continu	Technique
Critère d'évaluation	Vélocité	Fluidité	Qualité
Gestion des risques	Détection	Visualisation	Prévention
Implication client	Ponctuelle	Variable	Très forte et continue

TABLE I.2 – Les principales méthodes Agiles.

3.3 Choix de la méthode Scrum

Scrum a été retenue comme cadre de travail pour structurer le développement en itérations courtes tout en s'adaptant aux changements fréquents. Cette méthode offre plusieurs avantages décisifs : – Collaboration continue entre l'équipe et les parties prenantes

- Adaptation rapide aux changements grâce aux cycles itératifs
- Livraisons régulières de fonctionnalités opérationnelles
- Détection précoce des problèmes et ajustements immédiats
- Transparence totale sur l'avancement du projet



FIGURE I.9 – les piliers de la méthode Scrum

4 Fondements de SCRUM

Scrum est une méthode agile structurant le développement en sprints, cycles courts de 2 à 4 semaines. Les besoins sont exprimés sous forme de user stories décrivant les fonctionnalités attendues

et leur priorité. Cette approche vise à maximiser la productivité tout en assurant la satisfaction client par des livraisons régulières.

La méthode repose sur trois piliers fondamentaux : la transparence garantit la visibilité sur l'avancement, l'inspection permet d'évaluer régulièrement le produit et le processus, et l'adaptation autorise les ajustements nécessaires en fonction des retours.

5 Rôles de SCRUM

L'équipe Scrum est auto-organisée et prend les décisions sur la manière dont le travail sera réalisé afin d'atteindre l'objectif du sprint. Elle est composée d'un propriétaire de produit, d'un Scrum Master et d'une équipe de développement :

Le **Product Owner** est l'expert du domaine métier du projet qui porte la vision du produit à réaliser. Ce rôle repose sur la représentation de la voix du client et veille à ce que le produit reste aligné sur la vision stratégique de l'organisation.

Le **Scrum Master** a pour objectif d'assurer la productivité de l'équipe dans l'application de Scrum. Sa collaboration avec l'équipe vise à maximiser la performance globale, en alignant les efforts sur les objectifs stratégiques.

L' **équipe de développement** est constituée des personnes chargées de la réalisation du sprint, ayant pour mission de transformer les besoins exprimés en fonctionnalités utilisables du produit.

6 Evènements Scrum

Ce sont des rendez-vous clés qui rythment le projet Scrum, créent une routine de transparence et d'inspection, et permettent à l'équipe de s'adapter pour livrer la meilleure valeur possible.

- **Sprint** : Un sprint est une itération de développement d'une durée fixe (généralement **2 à 4 semaines**) durant laquelle une version potentiellement livrable du produit est réalisée.
- **Sprint Planning** : Réunion de planification du sprint où l'équipe définit les objectifs et sélectionne les éléments du Product Backlog à réaliser pendant le sprint.
- **Daily Scrum** : Réunion quotidienne de **15 minutes** permettant à l'équipe de synchroniser ses actions, d'identifier les obstacles et d'ajuster le plan de travail.
- **Revue de Sprint** : Réunion à la fin du sprint pour présenter le travail accompli, recueillir les retours et adapter le Product Backlog si nécessaire.
- **Sprint Rétrospective** : Réunion d'amélioration continue où l'équipe analyse le déroulement du sprint, identifie ce qui a bien fonctionné et ce qui doit être amélioré.

7 Artéfacts Scrum

Ce sont les supports d'information qui matérialisent le travail, la valeur et l'avancement du projet, garantissant une transparence totale entre l'équipe et les parties prenantes.

- **Product Backlog** : Liste priorisée de toutes les fonctionnalités, évolutions, corrections et tâches à réaliser pour le produit.
- **Sprint Backlog** : Sous-ensemble du Product Backlog sélectionné pour le sprint en cours, accompagné d'un plan pour livrer l'incrément.
- **Incrément** : Somme de tous les éléments du Product Backlog terminés durant un sprint et les sprints précédents, représentant une version utilisable du produit.

8 Cycle de vie

Le cycle de vie Scrum se compose de sprints successifs, chacun comprenant une planification, le développement, des réunions quotidiennes, une revue et une rétrospective. Ce processus itératif permet d'adapter le produit en continu aux besoins du client et d'améliorer l'équipe à chaque itération.

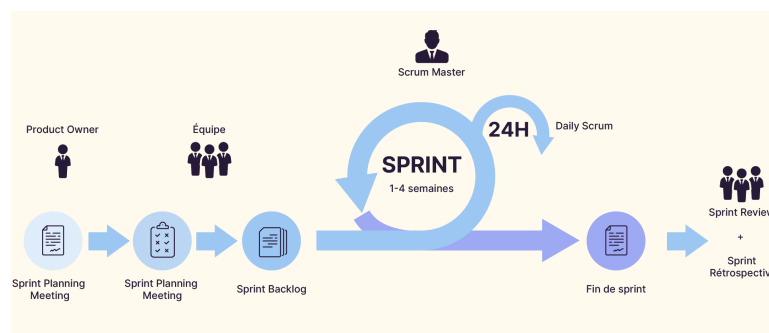


FIGURE I.10 – Cycle de vie Scrum

IV Langage de modélisation UML – Unified Modeling Language

« Le langage de modélisation unifié (UML) est un langage permettant de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels, ainsi que de modéliser des systèmes métier et d'autres systèmes non logiciels. L'UML représente un ensemble de bonnes pratiques d'ingénierie qui ont fait leurs preuves dans la modélisation de systèmes vastes et complexes. » [4]



FIGURE I.11 – Logo UML

Pour concevoir notre application, nous avons choisi UML comme langage de modélisation. Ce choix s’appuie sur plusieurs points forts tels que la communication entre l’équipe et la cohérence architecturale qui se fait de manière simplifiée et standardisée.

Conclusion

Ce chapitre a établi le cadre général de notre projet **SolidMaint** en présentant l’organisme d’accueil SolidWall Consulting et en analysant les enjeux de la gestion des contrats de maintenance. L’étude critique de l’existant a révélé des limitations significatives dans les processus actuels, tandis que l’analyse comparative des solutions du marché a confirmé la pertinence de développer une solution sur mesure. La solution **SolidMaint** proposée répond à ces défis par une approche intégrée centralisant l’ensemble du cycle de vie des contrats. L’adoption de la méthodologie Scrum et la planification en quatre sprints garantissent une approche itérative et collaborative pour la réalisation du projet. Le chapitre suivant détaillera la spécification des besoins et la modélisation de la solution.

Références

- [1] SolidWall Consulting. Site officiel. Disponible sur : <https://solidwall.com.tn/> (Consulté le 21/04/2026)
- [2] SolidWall Consulting. Publication LinkedIn PFE 2025-2026. Disponible sur : https://www.linkedin.com/posts/solid-wall-consulting_pfe-book-2025-2026-activity-7402287392465862656-MR2f (Consulté le 21/04/2026)
- [3] Atlassian. Qu’est-ce que la méthodologie Agile ? Disponible sur : <https://www.atlassian.com/fr/agile> (Consulté le 21/04/2026)
- [4] OMG. Why is UML important ? Disponible sur : <https://www.omg.org/uml/why-u>

[ml-is-important.htm](#) (Consulté le 21/04/2026)

[5] Miro. Qu'est-ce qu'un diagramme de Gantt ? Disponible sur : <https://miro.com/fr/planning-strategique/qu-est-ce-qu-un-diagramme-de-gantt/> (Consulté le 21/04/2026)

[6] Openfire. (s.d.). Contrat de maintenance : comment simplifier la gestion de vos contrats d'entretien annuels ? Disponible sur : <http://openfire.fr/blog/toutes-actualites-3/contrat-de-maintenance-comment-simplifier-la-gestion-de-vos-contrats-dentretien-annuels-74> (Consulté le 21/04/2026)

[7] Freshworks. Freshdesk - Customer Support Software. Disponible sur : <https://www.freshworks.com/freshdesk/> (Consulté le 21/04/2026)

[9] Atlassian. Jira Service Management. Disponible sur : <https://www.atlassian.com/software/jira/service-management> (Consulté le 21/04/2026)

[11] AXELOS. ITIL 4 Foundation - IT Service Management. Disponible sur : <https://www.axelos.com/certifications/itil-service-management> (Consulté le 21/04/2026)



CHAPITRE 2

Préparation du projet



Sommaire

2

I	Spécification des besoins	20
II	Modélisation des besoins	26
III	Pilotage du projet avec Scrum	28
IV	Environnement de Développement	52
V	Architecture de l'application	55
VI	Conclusion	56

Introduction

Ce chapitre présente l'analyse et la spécification des besoins de notre plateforme **SolidMaint**. Nous identifierons les acteurs du système et leurs besoins fonctionnels et non fonctionnels, puis procéderons à la modélisation UML. Nous détaillerons ensuite l'approche de pilotage Scrum avec la planification des sprints, et présenterons l'environnement de développement ainsi que l'architecture technique de l'application.

I Spécification des besoins

L'étape de spécification des besoins est importante pour prouver qu'un besoin réel existe, elle permet de définir explicitement les objectifs à atteindre.

Pour cela nous allons identifier les acteurs, leurs besoins fonctionnels et les besoins non fonctionnels.

1 Identification des acteurs

Un acteur représente une personne, une organisation ou un autre système qui interagit avec le système. Notre projet identifie six acteurs principaux :

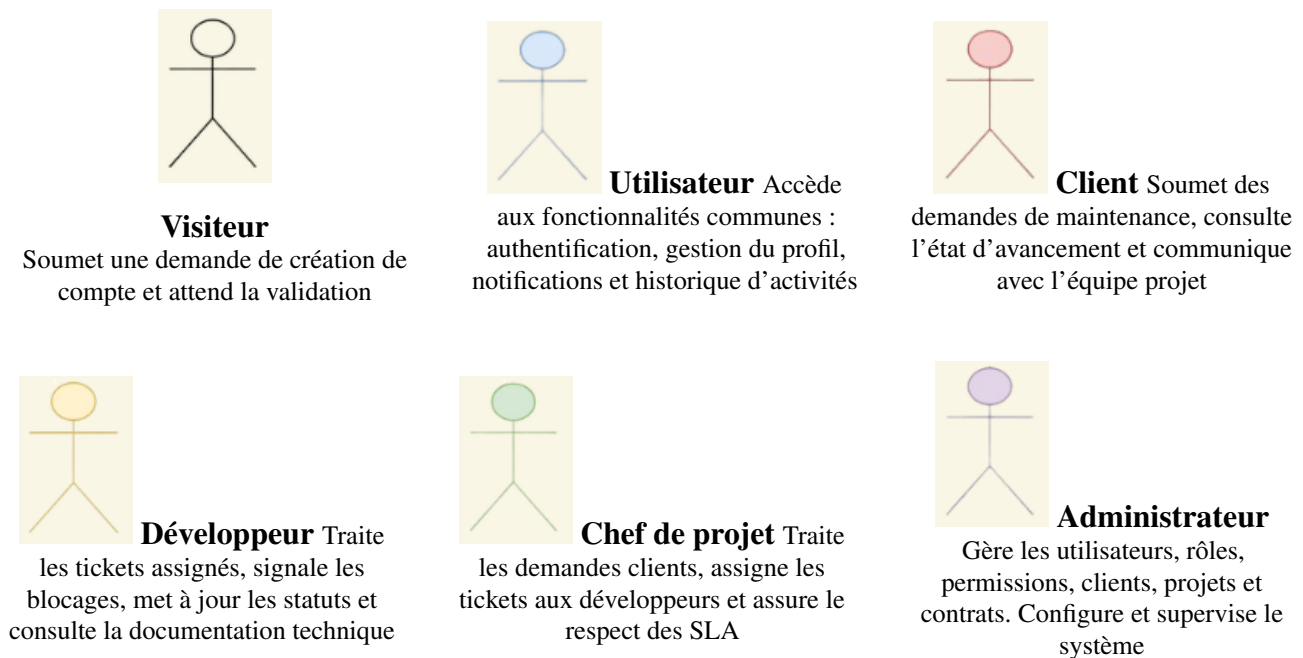


FIGURE II.1 – Les six acteurs du système SolidMaint et leurs responsabilités principales

2 Identification des besoins fonctionnels

L'identification des besoins fonctionnels constitue une étape fondamentale dans la conception de **SolidMaint**. Ces spécifications définissent précisément les opérations que la plateforme doit être capable d'exécuter pour répondre aux exigences métier de SolidWall Consulting.

2.1 Authentification et Gestion de Compte

Cette composante assure l'authentification sécurisée via identifiants classiques ou OAuth 2.0 (Google, GitHub) avec gestion automatique des jetons d'accès. L'innovation distinctive réside dans l'obligation de changement de mot de passe lors de la première connexion pour les comptes créés par un administrateur. Une interface modale bloque l'accès à l'application jusqu'à la définition d'un mot de passe personnel, garantissant que seul l'utilisateur légitime détient les informations d'authentification définitives.

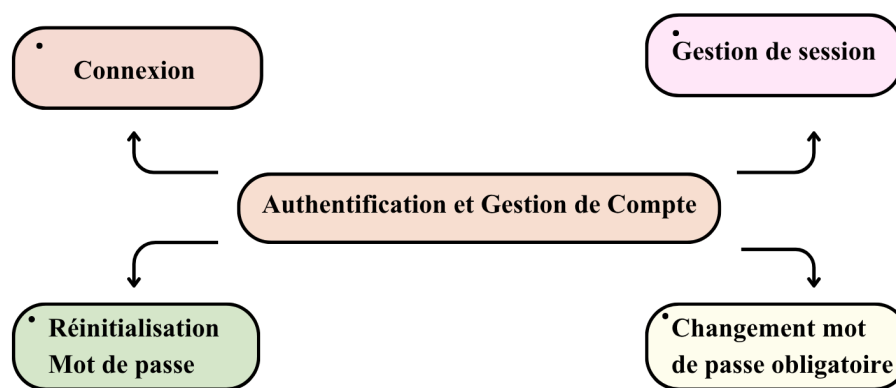


FIGURE II.2 – Authentification et Gestion de Compte

2.2 Gestion des Utilisateurs et Demandes d'Inscription

Ce système permet aux administrateurs de gérer le cycle de vie complet des comptes utilisateurs avec création automatisée, gestion des profils et contrôle des activations/désactivations. L'innovation principale réside dans le mécanisme de recherche et filtrage multicritères avancé qui permet de localiser rapidement un utilisateur spécifique selon des critères combinés (nom, rôle, statut, date de création). Le traitement des demandes d'inscription s'effectue via une interface centralisée permettant approbation, rejet ou mise en attente avec notifications automatiques.

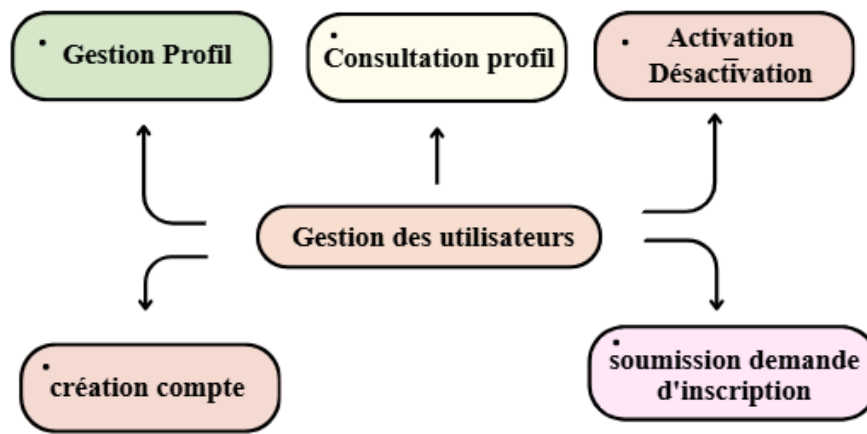


FIGURE II.3 – Gestion des utilisateurs

2.3 Système de Rôles et Permissions

Le contrôle d'accès s'appuie sur un modèle RBAC (Role-Based Access Control) offrant une granularité fine dans la gestion des autorisations. L'innovation majeure réside dans la conception modulaire permettant aux administrateurs de créer des rôles personnalisés en combinant dynamiquement les permissions existantes selon les besoins organisationnels spécifiques. Cette flexibilité permet d'ajuster les autorisations d'un rôle sans impact sur les autres configurations, facilitant l'évolution du système de sécurité.

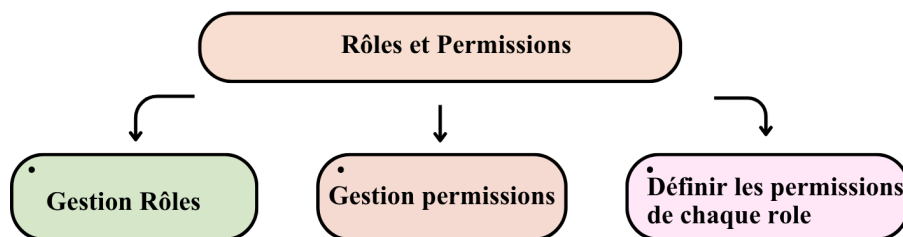


FIGURE II.4 – Rôles et permissions

2.4 Gestion des Clients et Projets

Cette composante centralise la gestion des clients avec création automatisée des comptes et récupération automatique des logos d'entreprise depuis les ressources en ligne. L'aspect distinctif est la génération automatique d'organigrammes visuels et interactifs pour chaque projet, présentant la

hiérarchie complète (client, chef de projet, équipe) avec accès direct aux coordonnées de tous les interlocuteurs. Cette visualisation facilite l'identification rapide des contacts et optimise la communication en cas de problème, tout en maintenant la traçabilité via des notes confidentielles horodatées.

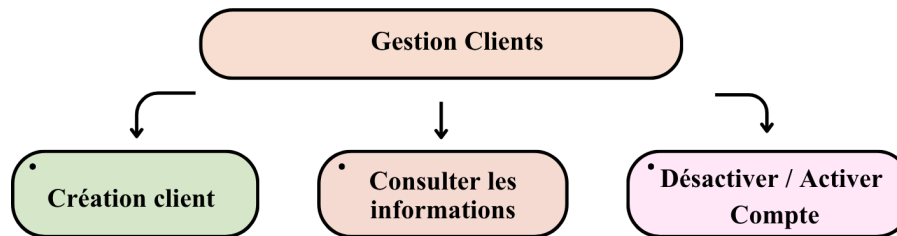


FIGURE II.5 – Gestion des Clients

2.5 Gestion des Contrats de Maintenance

Ce mécanisme automatise le cycle de vie complet des contrats de maintenance avec système d'alertes préventives échelonnées (30, 15, 7 jours avant expiration). L'innovation majeure réside dans l'import intelligent à partir de fichiers PDF, utilisant l'extraction automatique de données pour pré-remplir les formulaires de création. Un dispositif de détection de doublons basé sur le calcul d'empreintes cryptographiques garantit l'unicité des contrats et prévient toute duplication accidentelle lors des téléversements.

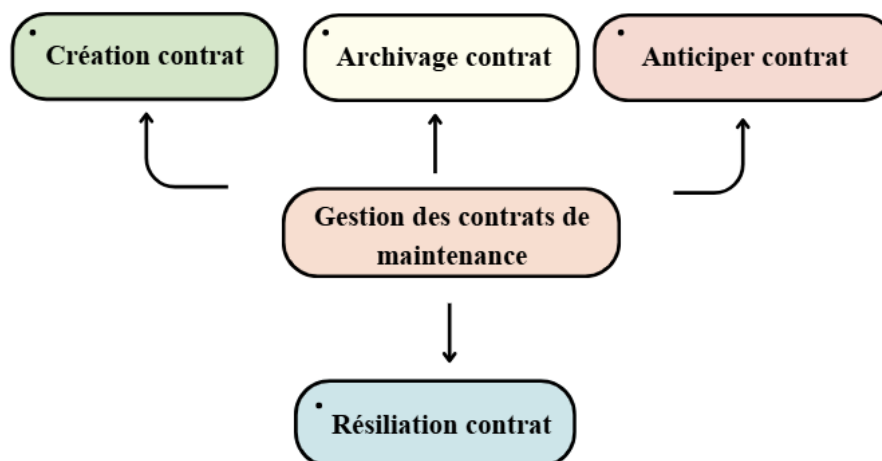


FIGURE II.6 – Gestion des contrats de maintenance

2.6 Gestion des Demandes d'Intervention

Ce processus structure la soumission des demandes d'intervention avec vérification automatique de couverture contractuelle et validation des types d'intervention éligibles. L'aspect distinctif impose

une contrainte temporelle intelligente de vingt-quatre heures entre deux demandes de même type pour un projet identique, prévenant la surcharge des équipes de support. Le workflow de validation intègre un système de relance automatique garantissant le traitement dans les délais impartis avec escalade vers les responsables en cas de dépassement des seuils temporels.

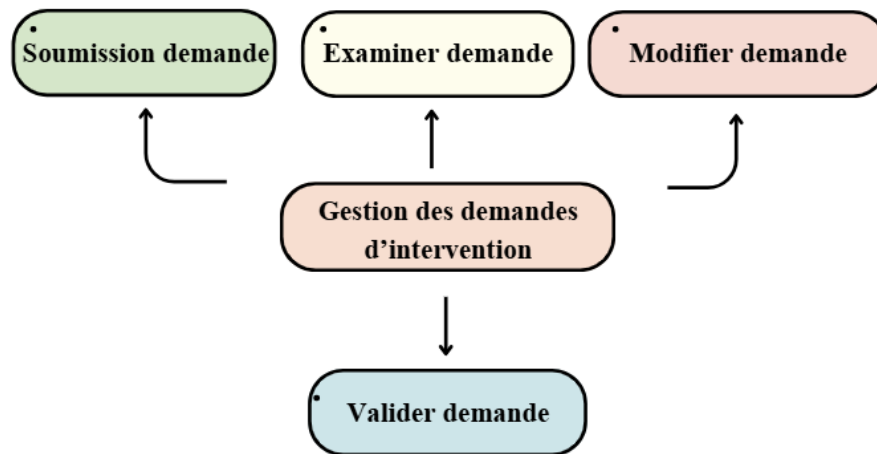


FIGURE II.7 – Gestion des demandes d'intervention

2.7 Gestion Avancée des Tickets de Maintenance

Ce système implémente un workflow structuré avec assignation intelligente limitée à cinq tickets actifs par développeur pour optimiser la charge de travail. L'innovation majeure est le mécanisme de rattachement de tickets complexes permettant de lier plusieurs interventions traitant du même contexte ou de problèmes dérivés. Les tickets rattachés héritent automatiquement du contrat parent et maintiennent un historique cohérent, facilitant la gestion des problèmes récurrents tout en préservant la traçabilité individuelle de chaque intervention.

2.8 Tableaux de Bord, Traçabilité et Documentation

Ce module offre des tableaux de bord personnalisés par rôle avec indicateurs clés de performance et visualisations analytiques en temps réel. Il maintient un historique exhaustif de toutes les opérations avec traçabilité complète (utilisateur, action, valeurs avant/après, horodatage) et système de notifications multicanaux (web, email, Slack). L'innovation majeure réside dans la génération automatique de procès-verbaux PDF à partir des informations de tickets (temps passé, description, solution), s'activant lors de la résolution pour produire automatiquement une documentation contractuelle structurée.

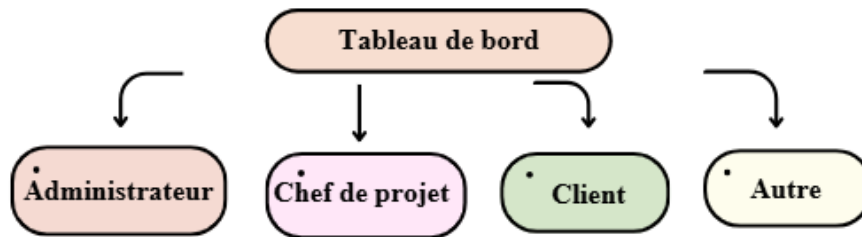


FIGURE II.8 – Tableau de bord et reportage

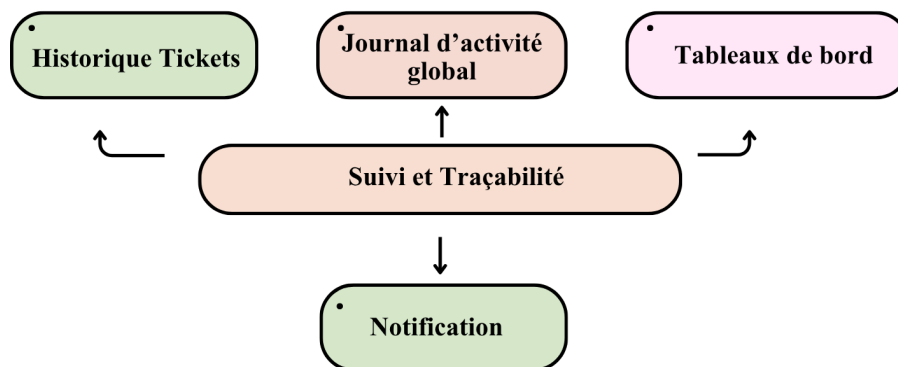


FIGURE II.9 – Suivi et Traçabilité

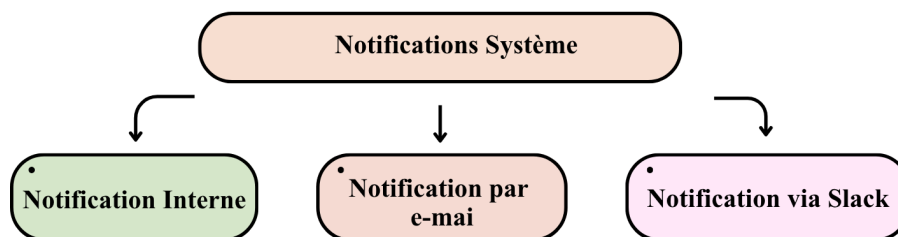


FIGURE II.10 – Notifications Système

3 Identification des besoins non fonctionnels

Durant la réalisation de ce projet, nous prenons en considération les besoins non fonctionnels qui désignent les attributs de qualité d'un système.

Les critères que notre plateforme doit satisfaire sont :

► Ergonomie :

L'interface intégrera **shadcn/ui** et **Tailwind CSS** pour assurer une cohérence visuelle et une utilisabilité optimale. Un système de thématisation dynamique (mode clair/sombre) et une architecture responsive garantiront une adaptation automatique aux différentes résolutions

d'écran. Les composants React permettront des mises à jour en temps réel sans rechargement, avec des transitions subtiles et un système de retour d'information clair pour guider l'utilisateur.

► **Sécurité :**

La sécurité intégrera des mesures robustes : hachage des mots de passe avec **bcrypt** (facteur 10), authentification JWT et OAuth 2.0 (Google, GitHub). Les protections incluront la validation contre les injections XSS, les requêtes paramétrées via **Prisma ORM** contre les injections SQL, et la vérification SHA-256 des fichiers téléversés. La journalisation complète des actions et la génération de mots de passe temporaires sécurisés renforceront la traçabilité et l'intégrité du système.

► **Performance :**

L'architecture modulaire permettra une évolution horizontale avec scalabilité selon la croissance des utilisateurs et du volume de données. L'optimisation de la base de données s'appuiera sur une stratégie d'indexation appropriée (identifiants, dates, statuts) et l'utilisation de **Prisma ORM** pour générer des requêtes SQL optimisées. Le chargement sélectif des relations via les mécanismes **select** et **include** évitera les problèmes de requêtes N+1, garantissant des temps de réponse rapides.

► **Maintenabilité :**

L'architecture modulaire et l'adoption des principes SOLID avec les conventions **Next.js** et **NestJS** garantiront la lisibilité et la facilité de modification du code. La documentation technique sera générée automatiquement via **Swagger/OpenAPI** et maintenue à jour lors de chaque modification. Une couverture de tests minimale de 80% sera assurée : tests unitaires avec **Jest**, tests d'intégration pour les API, et tests end-to-end avec **Playwright**, exécutés automatiquement avant chaque déploiement.

II Modélisation des besoins

L'analyse des besoins fonctionnels se poursuit par une modélisation dynamique à l'aide du diagramme de cas d'utilisation UML. Ce diagramme offre une vision synthétique des fonctionnalités offertes par le système du point de vue de chaque acteur, sans se préoccuper de leur ordre d'exécution ou des détails techniques.

La figure II.11 présente le diagramme de cas d'utilisation global de notre plateforme de service desk. Ce diagramme structure les interactions entre les cinq acteurs identifiés et le système. Chaque acteur possède un périmètre d'actions bien défini.

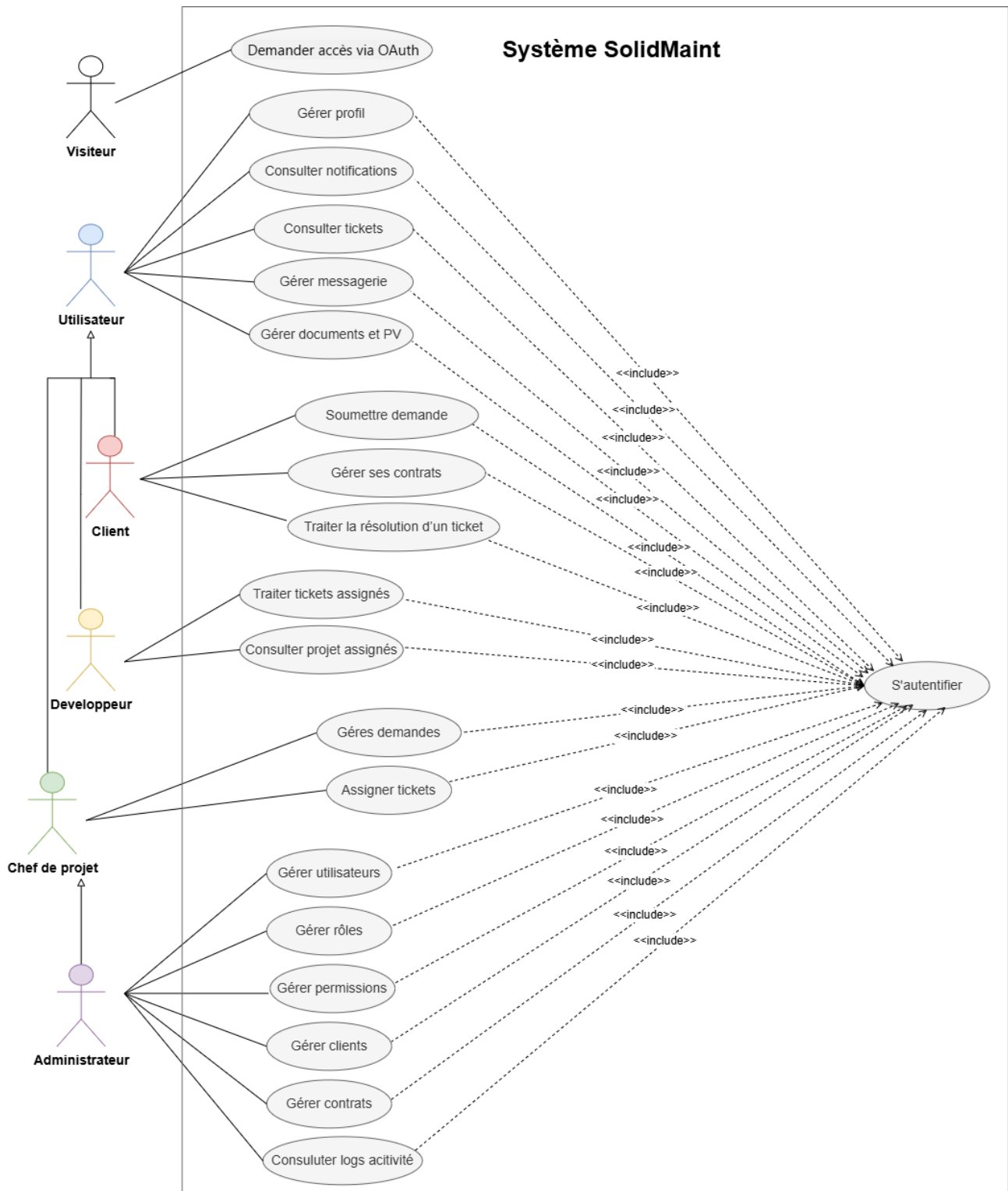


FIGURE II.11 – Diagramme de cas d'utilisation global du système

III Pilotage du projet avec Scrum

1 Rôles de SCRUM

Dans notre projet, Monsieur Mejdi Dhokar sera le Product Owner, Madame Maissa Ben Fredj sera le Scrum Master, et nous, Rihab Chourou et Eya Gharbi, formons l'équipe de développement.



FIGURE II.12 – Équipe scrum et rôles

1.1 Backlog de produit

Comme évoqué dans le premier chapitre, le Product Backlog constitue l'un des artefacts fondamentaux de la méthodologie Scrum. Il permet de formaliser les exigences fonctionnelles ainsi que les fonctionnalités attendues du système. Le tableau ci-dessous présente le Product Backlog de notre solution **SolidMaint**, structuré selon les éléments suivants :

- **Id EPIC** : Identifiant unique attribué à chaque EPIC afin de faciliter son identification et sa gestion dans le système de projet.
- **EPIC** : Ensemble de tâches interconnectées visant à atteindre un objectif majeur dans la gestion agile de projet ou le développement logiciel.
- **Id User Story** : Valeur unique et incrémentale associée à chaque user story pour assurer sa traçabilité.
- **User Story** : Description de la fonctionnalité du point de vue de l'utilisateur final, suivant le format "En tant que [rôle], je souhaite [action] afin de [bénéfice]".
- **Critères d'acceptation** : Conditions spécifiques et mesurables qui doivent être remplies pour que la user story soit considérée comme terminée et acceptée par le Product Owner.
- **Priorité** : Niveau d'importance défini par le Product Owner pour chaque tâche, permettant de prioriser les développements.

Légende des priorités : + : Faible, ++ : Moyenne, +++ : Élevée.

ID Epic	Épic	ID US	User Story	Critères d'acceptation	Priorité
1	Authentification et Sécurité	US01	En tant qu' utilisateur , je souhaite m'authentifier avec email et mot de passe afin d'accéder à mes informations de manière sécurisée.	L'utilisateur authentifié avec succès accède à son tableau de bord personnalisé selon son rôle.	+++
		US02	En tant qu' utilisateur , je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	L'utilisateur reçoit un email avec un lien valide 24h et peut définir un nouveau mot de passe conforme aux règles de sécurité.	+++
		US03	En tant qu' utilisateur , je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	L'utilisateur peut se connecter via son compte Google ou GitHub et accéder immédiatement à l'application.	++
		US04	En tant qu' utilisateur , je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	Après déconnexion, l'utilisateur ne peut plus accéder aux pages protégées sans se reconnecter.	++
		US05	En tant qu' utilisateur , je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	Après 3 tentatives échouées, le compte est temporairement verrouillé et l'utilisateur reçoit une notification par email.	+++
2	Gestion des utilisateurs et profils	US06	En tant qu' utilisateur , je souhaite gérer mon profil afin que mes informations soient toujours à jour.	L'utilisateur peut modifier et sauvegarder ses informations personnelles qui sont validées avant enregistrement.	++

		US07	En tant qu' administrateur , je souhaite créer des utilisateurs afin de gérer les accès au système.	Le nouvel utilisateur reçoit un email d'invitation avec un mot de passe temporaire pour sa première connexion.	+++
		US08	En tant qu' administrateur , je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et l'utilisateur concerné est notifié des changements.	++
		US09	En tant qu' administrateur , je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	Un compte désactivé ne peut plus se connecter et l'utilisateur reçoit une notification du changement de statut.	+++
		US10	En tant qu' administrateur , je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	La liste complète des utilisateurs est affichée avec possibilité de filtrer, rechercher et exporter les données.	++
		US11	En tant qu' utilisateur , je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	L'utilisateur visualise toutes ses actions passées avec possibilité de filtrer par date et type d'action.	++
3	Gestion des rôles et permissions	US12	En tant qu' administrateur , je souhaite gérer des rôles afin de structurer les accès au système.	L'administrateur peut créer, modifier et archiver des rôles avec vérification des utilisateurs associés.	+++
		US13	En tant qu' administrateur , je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	L'administrateur sélectionne les permissions par catégorie et visualise un aperçu avant validation.	+++

		US14	En tant qu' administrateur , je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	La matrice affiche clairement tous les rôles et leurs permissions avec possibilité de filtrer et exporter.	++
4	Gestion des demandes d'inscription OAuth	US15	En tant qu' utilisateur externe , je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	La demande est enregistrée et l'administrateur reçoit une notification pour traitement.	++
		US16	En tant qu' administrateur , je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	L'utilisateur reçoit un email de confirmation ou rejet et peut accéder à l'application si validé.	+++
5	Gestion des clients	US17	En tant qu' administrateur , je souhaite créer un profil client afin de centraliser ses informations.	Le profil client est créé avec toutes les informations nécessaires et un identifiant unique est généré.	+++
		US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	La liste complète des clients est accessible avec possibilité de filtrer, rechercher et exporter.	++
		US19	En tant qu' administrateur , je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	Toutes les informations du client sont affichées incluant contrats, projets et historique des interactions.	++
		US20	En tant qu' administrateur , je souhaite modifier les informations d'un client afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et le client est notifié si ses coordonnées changent.	++

		US21	En tant qu' administrateur , je souhaite activer et désactiver un compte client afin de contrôler l'accès.	Un compte client désactivé ne peut plus accéder au système et reçoit une notification du changement.	+++
		US22	En tant qu' administrateur , je souhaite ajouter des notes privées sur un client afin de conserver des informations internes.	Les notes sont enregistrées avec date et auteur, visibles uniquement par l'équipe interne.	+
		US23	En tant qu' administrateur , je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	Un document PDF professionnel contenant toutes les informations du client est généré.	+
6	Gestion des projets et équipes	US24	En tant qu' administrateur , je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	Le projet est créé et rattaché au client avec un chef de projet assigné.	+++
		US25	En tant qu' administrateur , je souhaite consulter la liste des projets afin de piloter l'activité.	La liste complète des projets est affichée avec possibilité de filtrer par client, statut ou chef de projet.	++
		US26	En tant qu' administrateur , je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	Les développeurs sont affectés au projet et peuvent visualiser les informations du projet.	+++
		US27	En tant que membre d'équipe , je souhaite consulter les informations du projet afin de comprendre le contexte.	Toutes les informations du projet sont accessibles incluant client, objectifs, équipe et contrat.	++

7	Gestion des contrats de maintenance	US28	En tant qu' administrateur , je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	Le contrat est créé avec client, dates, type SLA, cycle facturation et configuration des alertes d'expiration.	+++
		US29	En tant qu' administrateur , je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	La liste affiche tous les contrats avec filtres, indicateurs visuels d'expiration et possibilité d'export.	++
		US30	En tant qu' administrateur , je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements.	Des alertes automatiques sont envoyées à J-30, J-15 et J-7 par email, in-app et Slack.	+++
		US31	En tant qu' administrateur , je souhaite archiver un contrat afin de conserver l'historique.	Le contrat est archivé avec un changement de statut, avec possibilité de restauration ultérieure.	++
		US32	En tant que client , je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	La demande de résiliation est créée avec motif et date souhaitée, l'administrateur reçoit une notification et peut traiter la demande.	+++
		US33	En tant qu' administrateur , je souhaite résilier un contrat afin de l'interrompre avant terme.	Le contrat est résilié avec date et motif, les tickets ouverts sont clôturés et le client est notifié.	+++
		US34	En tant qu' administrateur , je souhaite renouveler un contrat afin de prolonger la relation.	Un nouveau contrat est créé à partir de l'existant avec lien entre les deux et notification au client.	+++

		US35	En tant que chef de projet , je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration.	Le calendrier affiche les contrats avec leur date d'expiration. Les contrats proches de l'expiration sont mis en évidence. L'administrateur peut exporter le calendrier.	++
		US36	En tant qu' administrateur , je souhaite centraliser les documents d'un contrat afin de les retrouver facilement.	L'administrateur peut téléverser des documents liés au contrat. Le système conserve l'historique des versions. Les documents sont accessibles depuis la fiche contrat.	+
		US37	En tant qu' administrateur , je souhaite configurer les règles SLA globales afin de définir les standards.	L'administrateur définit des délais de réponse selon l'urgence. L'administrateur définit des délais de résolution selon l'urgence. Les délais configurés s'appliquent automatiquement aux nouveaux tickets.	+++
8	Gestion des demandes clients	US38	En tant que client , je souhaite soumettre une demande de maintenance afin de signaler un besoin.	Le client peut joindre des fichiers à sa demande. Le système génère un numéro unique de suivi. Le client reçoit une confirmation de réception.	+++

		US39	En tant que chef de projet , je souhaite consulter les demandes en attente afin de les traiter.	Le chef de projet visualise toutes les demandes nécessitant un traitement. Les demandes sont triées par date de création. Les demandes urgentes sont identifiables visuellement.	+++
		US40	En tant que chef de projet , je souhaite valider une demande afin de la convertir en ticket.	Le chef de projet peut approuver une demande. Le chef de projet définit la catégorie et la priorité. Un ticket est automatiquement créé et lié à la demande.	+++
		US41	En tant que chef de projet , je souhaite demander des informations complémentaires afin de clarifier une demande.	Le chef de projet peut solliciter des précisions au client. Le client est notifié de la demande d'informations. Un rappel est envoyé si aucune réponse après 48h.	++
		US42	En tant que client , je souhaite compléter ma demande afin de fournir les informations manquantes.	Le client peut répondre aux questions posées. Le chef de projet est notifié de la réponse. La demande redevient disponible pour traitement.	++
		US43	En tant que chef de projet , je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Le chef de projet peut modifier la catégorie d'une demande. Le chef de projet peut ajuster la priorité. Une justification est obligatoire pour toute modification. Toutes les modifications sont tracées.	++

		US44	En tant qu' utilisateur , je souhaite commenter une demande afin de communiquer sur le contexte.	L'utilisateur peut ajouter des commentaires à une demande. L'utilisateur peut joindre des fichiers aux commentaires. Les autres parties prenantes sont notifiées des nouveaux commentaires.	++
		US45	En tant qu' utilisateur , je souhaite consulter l'historique d'une demande afin de suivre son évolution.	L'utilisateur visualise tous les événements liés à la demande. L'historique affiche la date et l'auteur de chaque action. L'utilisateur peut filtrer l'historique par type d'événement.	++
9	Gestion des tickets	US46	En tant que développeur , je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Le développeur visualise ses tickets organisés par état d'avancement. Le développeur peut déplacer un ticket d'un état à un autre. Le développeur voit si un ticket risque de dépasser son délai.	+++
		US47	En tant que chef de projet , je souhaite assigner un ticket à un développeur afin de répartir la charge.	Le chef de projet sélectionne un développeur disponible. Le système alerte si le développeur a déjà trop de tickets. Le développeur assigné est notifié immédiatement.	+++

		US48	En tant que chef de projet , je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Le chef de projet peut assigner plusieurs développeurs au même ticket. Le chef de projet définit le rôle de chaque développeur. Le temps estimé est réparti entre les développeurs.	++
		US49	En tant que développeur , je souhaite me désassigner d'un ticket afin de libérer ma charge.	Le développeur doit fournir un motif pour se désassigner. Le ticket redevient disponible pour assignation. Le chef de projet est notifié de la désassignation.	++
		US50	En tant que développeur , je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Le développeur peut démarrer le suivi du temps de travail. Une session de travail est créée avec l'heure de début. Le développeur voit le temps écoulé en temps réel.	+++
		US51	En tant que développeur , je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Le développeur peut suspendre temporairement le suivi du temps. Le temps écoulé est enregistré. Le développeur peut reprendre le suivi ultérieurement.	++
		US52	En tant que développeur , je souhaite arrêter le chronomètre afin de finaliser ma session.	Le développeur peut arrêter le suivi du temps. Le développeur peut ajouter un commentaire sur le travail effectué. La session est ajoutée à l'historique. Le développeur est alerté si le temps dépasse l'estimation.	+++

		US53	En tant que développeur , je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Le développeur peut enregistrer une session de travail passée. Le développeur doit fournir la date, l'heure et la durée. Le développeur doit justifier l'ajout manuel. Le système vérifie qu'il n'y a pas de chevauchement avec d'autres sessions.	++
		US54	En tant que développeur , je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Le développeur peut faire progresser un ticket selon le workflow défini. Le système empêche les transitions invalides. Toutes les parties prenantes sont notifiées lors d'un changement. L'historique des changements est conservé.	+++
		US55	En tant que client , je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Le client peut confirmer que le problème est résolu. Le client peut ajouter un commentaire de validation. Le ticket est fermé après validation. Le développeur est notifié de la validation.	+++
		US56	En tant que chef de projet , je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Le chef de projet doit justifier la réouverture. Le délai SLA est réinitialisé. Les développeurs assignés sont notifiés.	++

		US57	En tant que chef de projet , je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Le chef de projet visualise le temps total passé sur chaque ticket. Le chef de projet voit le détail par développeur. Les données sont présentées sous forme de graphiques.	++
		US58	En tant que développeur , je souhaite signaler un blocage afin d'alerter sur un obstacle.	Le développeur peut signaler un obstacle bloquant. Le développeur précise le type et l'impact du blocage. Le chef de projet reçoit une notification urgente. Le délai SLA est suspendu pendant le blocage.	+++
		US59	En tant que membre d'équipe , je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Le blocage est résolu avec commentaire obligatoire, le chrono SLA reprend et l'action est tracée.	++
		US60	En tant que développeur , je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	La demande d'extension est créée avec durée et justification pour validation par le chef de projet.	++
		US61	En tant que chef de projet , je souhaite approuver ou rejeter les demandes d'extension SLA.	La liste des demandes en attente est affichée, l'approbation ou rejet avec commentaire met à jour automatiquement le SLA.	++
		US62	En tant qu' utilisateur , je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Les commentaires sont ajoutés avec éditeur riche, autocomplétion @utilisateur et distinction internes/externes.	+++

		US63	En tant qu' utilisateur , je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Les fichiers sont ajoutés par drag & drop (images/PDF/logs, max 10MB) avec prévisualisation.	++
		US64	En tant que développeur , je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	Une notification temps réel via WebSocket affiche un toast et un indicateur "X est en train d'éditer".	++
		US65	En tant qu' utilisateur , je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	La timeline chronologique inversée affiche tous les événements avec filtres par type et export PDF.	++
10	Messagerie et collaboration	US66	En tant qu' utilisateur , je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	L'utilisateur visualise les canaux de discussion disponibles. L'utilisateur voit le nombre de messages non lus. L'utilisateur identifie les membres présents.	++
		US67	En tant qu' utilisateur , je souhaite envoyer des messages dans un channel afin de communiquer.	L'utilisateur peut rédiger et envoyer des messages. L'utilisateur peut joindre des fichiers. Les messages sont reçus instantanément par les autres membres.	+++
		US68	En tant qu' utilisateur , je souhaite répondre à un message afin de créer un fil de discussion.	L'utilisateur peut répondre à un message spécifique. Les réponses sont regroupées dans un fil dédié. Seuls les participants au fil reçoivent les notifications.	++

		US69	En tant qu' utilisateur , je souhaite mentionner des collègues afin d'attirer leur attention.	L'utilisateur peut mentionner un collègue dans un message. Le collègue mentionné reçoit une notification. Le message mentionné est mis en évidence pour le destinataire.	+++
		US70	En tant qu' utilisateur , je souhaite éditer mes messages afin de corriger des erreurs.	L'utilisateur peut modifier un message récemment envoyé. Le message modifié est identifiable. L'historique des modifications est conservé.	+
		US71	En tant qu' utilisateur , je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	L'utilisateur peut supprimer ses propres messages. Une confirmation est demandée avant suppression. Le message supprimé reste visible pour l'audit.	+
		US72	En tant qu' utilisateur , je souhaite épingler des messages importants afin de les retrouver facilement.	L'utilisateur autorisé peut épingler un message. Les messages épinglés sont facilement accessibles. L'utilisateur peut retirer l'épinglage.	++
		US73	En tant que chef de projet , je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	Le chef de projet visualise les membres actuellement connectés. Le chef de projet peut filtrer par équipe ou projet. Le chef de projet voit l'heure de dernière activité.	++

		US74	En tant qu' utilisateur , je souhaite voir qui est en ligne afin de savoir qui est disponible.	L'utilisateur voit le statut de disponibilité des autres membres. L'utilisateur peut modifier son propre statut. Le statut est visible par tous les membres.	++
11	Gestion des documents	US75	En tant qu' utilisateur , je souhaite uploader des documents afin de centraliser les fichiers.	L'utilisateur peut téléverser des documents. L'utilisateur peut associer un document à un contrat, projet ou ticket. L'utilisateur peut ajouter un titre, une description et des tags.	++
		US76	En tant qu' utilisateur , je souhaite télécharger des documents afin de les consulter hors ligne.	L'utilisateur peut télécharger un document individuel. L'utilisateur peut télécharger plusieurs documents simultanément. Le système vérifie les permissions avant le téléchargement. La consultation est enregistrée pour traçabilité.	++
		US77	En tant qu' utilisateur , je souhaite gérer les versions des documents afin d'assurer la traçabilité.	L'utilisateur peut téléverser une nouvelle version d'un document existant. Le système numérote automatiquement les versions. L'utilisateur doit fournir un commentaire pour chaque nouvelle version.	++

		US78	En tant qu' utilisateur , je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	L'utilisateur visualise qui a consulté le document. L'utilisateur voit la date et l'heure de chaque consultation.	+
		US79	En tant qu' utilisateur , je souhaite archiver des documents afin de nettoyer la vue active.	L'utilisateur peut archiver un document. Une confirmation est demandée avant archivage. Le document archivé peut être restauré ultérieurement.	+
		US80	En tant qu' utilisateur , je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Le système détecte les fichiers identiques lors du téléversement. L'utilisateur est alerté si un doublon existe. L'utilisateur peut choisir de remplacer, créer une version ou annuler.	++
		US81	En tant que chef de projet , je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Le chef de projet peut générer un procès-verbal depuis un ticket. Le document contient toutes les informations du ticket et les actions réalisées. Le document peut être signé électroniquement.	+++
		US82	En tant que client , je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Le client peut accéder aux procès-verbaux depuis ses tickets. Le client visualise l'historique de tous les PV générés. Le client reçoit une notification lors de la génération d'un nouveau PV.	++

12	Notifications et alertes	US83	En tant qu' utilisateur , je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	L'utilisateur visualise le nombre de notifications non lues. L'utilisateur peut consulter la liste des notifications. L'utilisateur peut marquer une notification comme lue. L'utilisateur peut accéder directement à l'élément concerné.	+++
		US84	En tant qu' utilisateur , je souhaite recevoir des notifications par email afin d'être informé hors connexion.	L'utilisateur peut configurer la fréquence des notifications email. L'utilisateur reçoit des emails pour les événements importants.	++
		US85	En tant que membre technique , je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	L'administrateur peut configurer l'intégration Slack par contrat ou projet. Les événements importants sont envoyés sur Slack.	++
		US86	En tant qu' utilisateur , je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	L'utilisateur peut autoriser les notifications navigateur. L'utilisateur reçoit des alertes pour les événements critiques. L'utilisateur peut accéder à l'application en cliquant sur la notification.	++
		US87	En tant qu' utilisateur , je souhaite consulter l'historique de mes notifications afin de retrouver une information.	L'utilisateur visualise toutes ses notifications passées. L'utilisateur peut filtrer par type ou période. L'utilisateur peut rechercher dans l'historique.	++

13	Dashboards et rapports	US88	En tant qu' administrateur , je souhaite consulter un dashboard global afin de piloter le système.	L'administrateur visualise les indicateurs clés du système. L'administrateur peut filtrer les données par période. L'administrateur identifie les alertes nécessitant une action.	+++
		US89	En tant que chef de projet , je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	Le chef de projet visualise la charge de travail de son équipe. Le chef de projet suit la performance par rapport aux délais. Le chef de projet compare le temps passé au temps estimé.	+++
		US90	En tant que développeur , je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Le développeur visualise ses tickets assignés. Le développeur voit ses priorités du jour. Le développeur suit son temps de travail. Le développeur identifie les échéances proches.	++
		US91	En tant que client , je souhaite consulter mon dashboard afin de suivre mes projets.	Le client visualise ses contrats actifs. Le client suit l'état de ses demandes et tickets. Le client accède à ses documents récents. Le client identifie ses contacts dans l'équipe.	++
		US92	En tant qu' administrateur , je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	L'administrateur génère un rapport pour une période donnée. Le rapport contient les tickets traités et le temps passé. Le rapport indique le respect des délais. Le rapport peut être exporté.	+++

		US93	En tant qu' administrateur , je souhaite générer des rapports de conformité SLA afin de justifier les performances.	L'administrateur génère un rapport de conformité par contrat. Le rapport affiche le taux de respect des délais. Le rapport détaille les tickets en dépassement.	+++
		US94	En tant que chef de projet , je souhaite générer des rapports d'activité afin de présenter l'avancement.	Le chef de projet génère un rapport pour son équipe ou projet. Le rapport affiche les tickets traités et le temps passé. Le rapport présente l'évolution de l'activité.	++
14	Historique et traçabilité	US95	En tant qu' administrateur , je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.	L'administrateur visualise toutes les actions effectuées dans le système. L'historique indique l'utilisateur, la date et l'action réalisée. L'historique conserve les valeurs avant et après modification.	+++
		US96	En tant qu' administrateur , je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.	L'administrateur peut exporter l'historique des actions. L'administrateur peut filtrer par période, utilisateur ou type d'action. Les données exportées sont sécurisées.	++
15	Versioning avancé	US97	En tant qu' utilisateur , je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	L'utilisateur peut téléverser une nouvelle version d'un document. L'utilisateur doit fournir un commentaire pour la nouvelle version. Le système numérote automatiquement les versions. Toutes les versions précédentes sont conservées.	++

		US98	En tant qu' utilisateur , je souhaite comparer deux versions d'un document afin de voir les différences.	L'utilisateur peut sélectionner deux versions à comparer. Les deux versions sont affichées côte à côte. Les modifications sont mises en évidence.	+
		US99	En tant qu' administrateur , je souhaite voir qui a consulté un document afin de suivre sa diffusion.	L'administrateur visualise la liste des consultations du document. L'historique indique la date, l'heure et l'utilisateur. Des statistiques de consultation sont disponibles.	++
16	Assistance intelligente	US100	En tant que client , je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Le système analyse le contenu de la demande. Le système propose une catégorie appropriée. Le client peut accepter ou modifier la suggestion.	+++
		US101	En tant qu' administrateur , je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Le système apprend des validations et rejets des suggestions. Le modèle se réentraîne périodiquement. La précision des suggestions est mesurée.	++
		US102	En tant que chef de projet , je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Le système analyse le contenu et le contexte de la demande. Le système propose un niveau de priorité. Le chef de projet peut ajuster la priorité suggérée.	++

		US103	En tant que développeur , je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Le système identifie les tickets similaires résolus. Le système suggère les solutions précédemment appliquées. Le développeur peut accepter ou ignorer les suggestions.	++
		US104	En tant que développeur , je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Le développeur peut noter la pertinence d'une suggestion. Les retours sont collectés pour améliorer le système. Le modèle s'ajuste en fonction des évaluations.	+
		US105	En tant qu' administrateur , je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	Le système analyse la similarité lors de la création d'une demande. Le système signale les demandes potentiellement identiques. Les demandes similaires sont automatiquement liées.	++

TABLE II.1 – Product Backlog

2 Planification des sprints

Un sprint représente une période de développement durant laquelle l'équipe s'engage à livrer un ensemble cohérent de fonctionnalités opérationnelles. Cette approche itérative permet d'obtenir des retours réguliers et d'ajuster les priorités au fil de l'avancement.

Notre projet est organisé en **4 sprints** de **3 à 4 semaines** chacun, couvrant une période totale de développement d'**environ 3 mois et demi**. La stratégie de répartition suit une logique architecturale : fondations sécuritaires (Sprint 1), gestion contractuelle (Sprint 2), cœur métier opérationnel (Sprint 3), et fonctionnalités avancées (Sprint 4). Chaque sprint commence par une réunion de planification (Sprint Planning) où l'équipe sélectionne les user stories du Product Backlog à réaliser, et se termine par une revue de sprint (Sprint Review) et une rétrospective (Sprint Retrospective).

La figure suivante présente le déroulement chronologique de nos sprints.

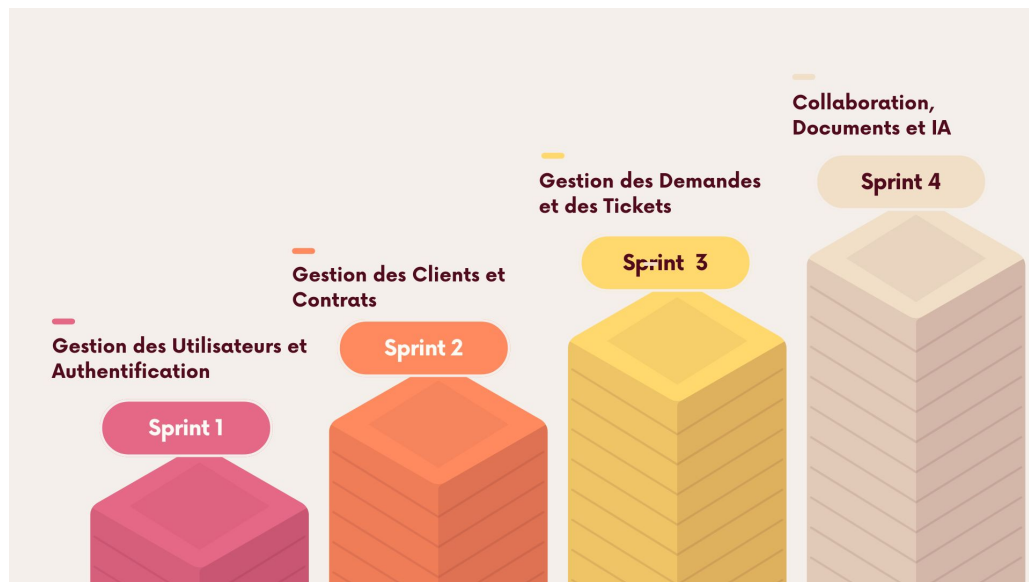


FIGURE II.13 – Planification chronologique des sprints

2.1 Sprint 1 - Gestion des Utilisateurs, Clients et Authentification

Période : du 17 février au 11 mars (3 semaines)

Objectif du sprint : Mettre en place le système d'authentification sécurisé, la gestion complète des utilisateurs et des clients avec contrôle d'accès basé sur les rôles (RBAC).

User Stories sélectionnées :

- **Epic 1 - Authentification et Sécurité :** US01 à US05 (5 user stories)
- **Epic 2 - Gestion des utilisateurs :** US06 à US11 (6 user stories)
- **Epic 3 - Gestion des rôles et permissions :** US12 à US14 (3 user stories)
- **Epic 4 - Demandes d'inscription OAuth :** US15 à US16 (2 user stories)
- **Epic 5 - Gestion des clients :** US17 à US23 (7 user stories)

Livrables attendus :

- Système d'authentification fonctionnel avec support OAuth
- Module de gestion des utilisateurs opérationnel
- Système RBAC complet avec gestion des rôles et permissions
- Module de gestion des clients avec profils société

2.2 Sprint 2 - Gestion des Projets et Contrats

Période : du 12 mars au 2 avril (3 semaines)

Objectif du sprint : Développer la gestion complète des projets et contrats de maintenance avec système d'alertes.

User Stories sélectionnées :

- **Epic 6 - Gestion des projets** : US24 à US27 (4 user stories)
- **Epic 7 - Gestion des contrats** : US28 à US37 (10 user stories)
- **Epic 14 - Historique et traçabilité** : US95 à US96 (2 user stories)

Livrables attendus :

- Module de gestion des projets
- Système de gestion des contrats avec SLA et alertes
- Système d'audit et traçabilité complet

2.3 Sprint 3 - Gestion des Demandes et Tickets

Période : du 3 avril au 1 mai (**4 semaines**)

Objectif du sprint : Implémenter le cœur métier avec la gestion des demandes et tickets incluant suivi du temps et SLA.

User Stories sélectionnées :

- **Epic 8 - Gestion des demandes** : US38 à US45 (8 user stories)
- **Epic 9 - Gestion des tickets** : US46 à US65 (20 user stories)
- **Epic 13 - Dashboards** : US88 à US91 (4 user stories)

Livrables attendus :

- Module de gestion des demandes de maintenance
- Système complet de gestion des tickets avec Kanban
- Suivi du temps et respect des SLA
- Dashboards pour pilotage

2.4 Sprint 4 - Collaboration, Documents et IA

Période : du 2 mai au 31 mai (**4 semaines**)

Objectif du sprint : Finaliser avec les fonctionnalités avancées de collaboration, gestion documentaire et IA.

User Stories sélectionnées :

- **Epic 10 - Messagerie** : US66 à US74 (9 user stories)
- **Epic 11 - Gestion des documents** : US75 à US82 (8 user stories)
- **Epic 12 - Notifications** : US83 à US87 (5 user stories)
- **Epic 13 - Rapports** : US92 à US94 (3 user stories)
- **Epic 15 - Versioning avancé** : US97 à US99 (3 user stories)
- **Epic 16 - Intelligence Artificielle** : US100 à US105 (6 user stories)

Livrables attendus :

- Système de messagerie et collaboration temps réel
- Module de gestion documentaire avec versioning
- Système de notifications multicanaux
- Module de génération de rapports
- Moteur d'IA pour assistance intelligente

Synthèse de la planification : Cette répartition assure une progression logique où chaque sprint s'appuie sur les livrables précédents. Les dépendances techniques sont respectées (authentification avant tickets, contrats avant demandes), et la valeur métier croît progressivement jusqu'aux fonctionnalités différenciantes du sprint final.

3 Planning prévisionnel (Diagramme de Gantt)

« Un diagramme de Gantt est un outil d'organisation et de gestion de projets. Il s'agit d'un diagramme à barres horizontales qui présente toutes les tâches d'un projet en fonction d'un calendrier. Il aide les chefs de projet à visualiser les tâches requises et à suivre l'avancement de leur projet. » [5]

Dans le cadre du développement de notre projet **SolidMaint**, le planning prévisionnel a été réalisé pour organiser les différentes phases de développement. La figure ci-dessous illustre cette planification :

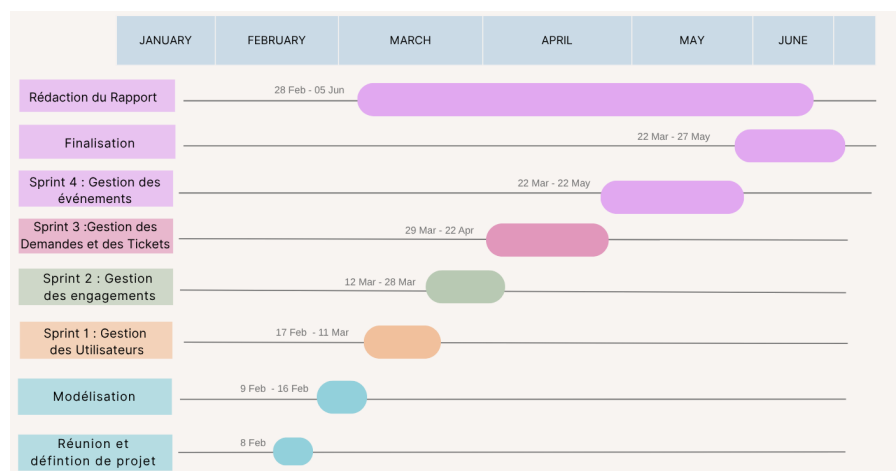


FIGURE II.14 – Diagramme de Gantt

IV Environnement de Développement

On va commencer par la présentation de l'environnement matériel et logiciel que nous utilisons.

1 Environnement matériel

Ordinateur	1	2
propriétés	Rihab Chourou	Eya Gharbi
Processeur	Core i3	Core i3
Ram	20 Go	16 Go
Disque Dure	1 To	256 Go
Système d'exploitation	Windows 10 Professionnel N 64-bit	Windows 10 Professionnel 64-bit

TABLE II.2 – Environnement de développement

2 Environnement logiciel

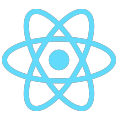
2.1 Langage de programmation et framework



NestJs V11.0.16 : « Cest un framework permettant de créer des applications serveur Node.js performantes et évolutives. Il utilise JavaScript progressif, est conçu avec TypeScript et le prend entièrement en charge (tout en permettant aux développeurs de coder en JavaScript pur). » |6|



Next V16.2.2 : « C'est un framework React permettant de créer des applications web complètes. Il configure également automatiquement les outils de bas niveau tels que les modules de regroupement et les compilateurs. » |7|



React 18.3.1 : « React est bien plus qu'une bibliothèque, une architecture ou même un écosystème. React est une communauté. C'est un lieu où vous pouvez demander de l'aide, trouver des opportunités et rencontrer de nouvelles personnes.. » |8|



Tailwind CSS V4.2.1 : « Un framework CSS utilitaire, doté de classes telles que flex , pt-4 , text-center et rotate-90, qui peuvent être combinées pour créer n'importe quel design, directement dans votre balisage. » |9|



shadcn/ui V3.8.5 : « est un ensemble de composants élégants et accessibles, ainsi qu’une plateforme de distribution de code. Compatible avec vos frameworks et modèles d’IA préférés. Logiciel libre. » |10|

2.2 Outils de développement et modélisation



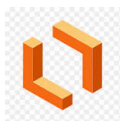
Visual Studio Code : « est un environnement de développement intégré (IDE) puissant pour Windows qui vous permet de développer, compiler, déboguer, tester et déployer vos applications, le tout au même endroit. » |11|



GitLab : « offre un espace de stockage de code en ligne ainsi que des fonctionnalités de suivi des problèmes et d’intégration continue/déploiement continu (CI/CD). Le dépôt permet d’héberger différentes chaînes de développement et versions, » |12|



UML : « est un langage permettant de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels, ainsi que de modéliser des systèmes métier et d’autres systèmes non logiciels. L’UML représente un ensemble de bonnes pratiques d’ingénierie qui ont fait leurs preuves dans la modélisation de systèmes vastes et complexes. » |13|



lucidchart : « est un environnement de travail visuel qui associe création de diagrammes, visualisation de données et fonctionnalités de collaboration afin de faciliter la communication et stimuler l’innovation. » |14|



Swagger : « Swagger permet la conception, la gouvernance et les tests tout au long du cycle de vie complet des API basées sur l’IA, garantissant ainsi la qualité à chaque étape. Concevoir des API adaptées aux humains, aux LLM, aux agents et à l’innovation continue. » |15|

2.3 Systèmes de Gestion de Bases de données



Prisma ORM V7.4.2 : « est un ORM open source qui offre un accès rapide et sécurisé aux bases de données Postgres, MySQL et autres, et fonctionne parfaitement sur Node.js. » |16|



PostgreSQL : « est un système de gestion de bases de données objet-relationnelles (SGBD-OR). Il a été pionnier dans de nombreux concepts qui ne sont devenus disponibles que bien plus tard dans certains systèmes de bases de données commerciaux. » |17|



Neon : « est un service de base de données PostgreSQL cloud natif, sans serveur et entièrement managé, conjointement développé et géré par Microsoft et Neon » |18|

2.4 Outil de Gestion de Projet Scrum



Notion : « est un puissant outil no-code de productivité. Il aide à gérer une base de connaissances, pour rendre plus facile et convivial le travail au sein des collaborateurs. Cette application se révèle indispensable autant pour les travailleurs autonomes que pour les professionnels en entreprise. C’est un également un outil parfait pour gérer son side project. » |18|

2.5 Environnement de composition de documents



LaTeX : « est un système de composition de haute qualité ; il comprend des fonctionnalités conçues pour la production de documents techniques et scientifiques. LaTeX est la norme de facto pour la communication et la publication de documents scientifiques. » |19|

2.6 Outil de design graphique



Canva : « est une plateforme de conception et de communication visuelle en ligne dont la mission est de permettre à chacun dans le monde de concevoir n’importe quoi et de publier n’importe où. » |20|

2.7 Outils de test et qualité logicielle



Playwright : « est un framework de test complet pour les applications web modernes. Il intègre un exécuteur de tests, des assertions, l’isolation et la parallélisation. Playwright est compatible avec Chromium, WebKit et Firefox sous Windows, Linux et macOS, en local ou en intégration continue, avec ou sans interface graphique, et propose une émulation mobile native pour Chrome (Android) et Safari mobile. » |21|

V Architecture de l'application

1 Architecture physique

L'architecture système est un ensemble de décisions de conception qui définit l'interaction des composants entre eux. Le choix de l'architecture est très sensible car elle permet d'optimiser les performances du système et facilite l'évolution du système en fonction des volumes de données. La figure ci-dessous montre l'architecture de notre application :

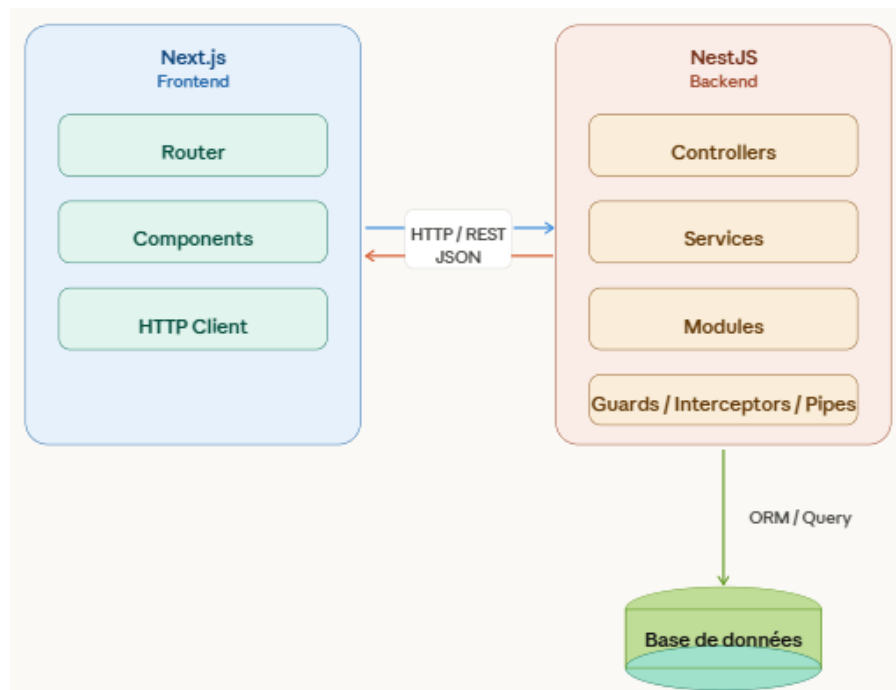


FIGURE II.15 – architecture physique

2 Architecture logique

Nous avons choisi comme architecture logicielle l'architecture du **monolithe modulaire**. Ce choix est basé sur la combinaison entre la simplicité et la robustesse des applications monolithiques traditionnelles.

L'architecture du **monolithe modulaire** nous permet de travailler dans une base de code unifiée, avec des frontières clairement définies et des modules indépendants. Elle consiste à diviser un système en plusieurs modules.

Nous avons choisi cette architecture afin de maintenir l'indépendance des modules et d'obtenir une grande cohésion, une encapsulation des données, ainsi qu'une focalisation sur les fonctionnalités métiers.

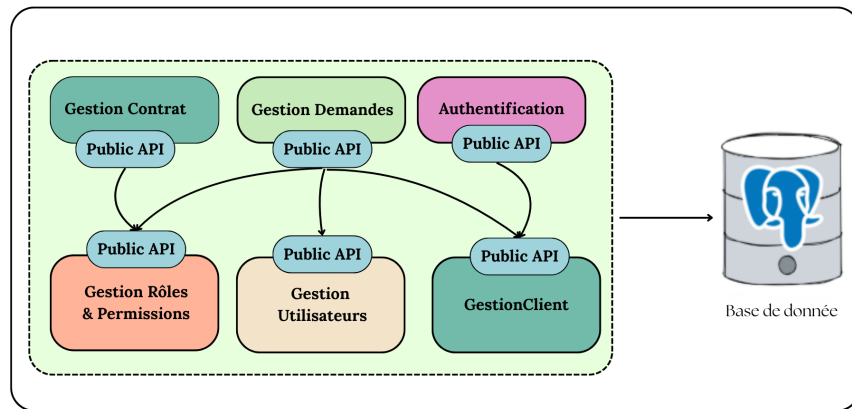


FIGURE II.16 – Architecture monolithique modulaire

VI Conclusion

Ce chapitre a établi les fondements techniques et méthodologiques de notre projet **SolidMaint**. L'identification des acteurs et la spécification des besoins fonctionnels et non fonctionnels ont permis de définir précisément les exigences de la plateforme. La modélisation UML et l'approche de pilotage Scrum avec la planification en quatre sprints structurent la démarche de développement. L'environnement de développement choisi et l'architecture modulaire garantissent une solution robuste et évolutive. Ces éléments constituent la base solide pour aborder la phase de réalisation.

Le chapitre suivant détaillera l'implémentation du premier sprint consacré à la gestion des utilisateurs et authentification.

Références

- **NestJS**, Documentation officielle - <https://docs.nestjs.com>
- **Next.js**, Documentation officielle - <https://nextjs.org/docs>
- **React**, Guide officiel - <https://react.dev>
- **Tailwind CSS**, Documentation - <https://tailwindcss.com>
- **shadcn/ui**, Guide des composants - <https://ui.shadcn.com/docs>
- **Visual Studio Code**, Manuel d'utilisation - <https://learn.microsoft.com/en-us/visual-studio/get-started/visual-studio-ide?view=visualstudio>
- **GitLab**, Guide utilisateur - <https://www.techtarget.com/whatis/definition/GitLab>
- **UML**, Spécifications officielles - <https://www.omg.org/uml/why-uml-is-important.htm>
- **draw.io**, Outil de modélisation - <https://www.lucidchart.com/pages/fr/exemple/logiciel-gestion-de-projet>
- **Swagger**, Documentation des APIs - <https://swagger.io/>
- **Prisma ORM**, Documentation - <https://www.prisma.io/docs>
- **PostgreSQL**, Site officiel - <https://www.postgresql.org>
- **Neon**, Cloud Postgres - <https://learn.microsoft.com/fr-fr/azure/partner-solutions/neon/overview>
- **Notion**, Espace de travail - <https://www.dixmilleheures.fr/avis/notion>
- **LaTeX**, Système de composition - <https://www.latex-project.org>
- **Canva**, Design graphique - <https://1password.com/fr/customers/canva>
- **Playwright**, Framework de test - <https://playwright.dev/docs/intro>

CHAPITRE 3

Gestion des Utilisateurs et Authentification

Sommaire



I	Sprint Planning Meeting	59
II	Analyse	65
III	Diagramme de classes de première sprint	84
IV	Schéma relationnel	84
V	Réalisation	85
VI	Tests et Validation	86

Introduction

Dans le chapitre précédent, nous avons défini les besoins liés à notre solution. Ensuite, nous avons réparti les différentes tâches du projet afin que les phases de travail soient correctement planifiées.

Au cours de ce chapitre, nous présenterons les fonctionnalités liées à la gestion des utilisateurs, retenues dans le cadre du premier sprint de notre projet.

I Sprint Planning Meeting

La planification du sprint est essentielle à la réussite de notre solution, au bout de laquelle on clarifie les différentes fonctionnalités à implémenter lors du sprint à venir. Cette partie est efficace afin de préparer des objectifs clairs ; Dans ce contexte, nous présentons l’objectif du sprint dans la section suivante.

1 Objectif du Sprint

L’objectif de ce sprint est de placer les tâches fonctionnelles et sécuritaires de la solution, en implémentant les mécanismes d’authentification et de sécurisation des accès ainsi que les fonctionnalités de gestion des utilisateurs et de leurs profils, et afin d’assurer un contrôle efficace des accès et des permissions en introduisant l’administration du système à travers la gestion des comptes utilisateurs et des rôles.

1.1 Backlog du premier sprint

Voici l’ensemble des fonctionnalités issues du Product Backlog, qui seront réalisées dans le cadre de ce sprint.

1 : Effort faible, 2 : Effort moyen, 3 : Effort élevé

ID US	User Story	Description technique	Effort
Epic 1 - Authentification et Sécurité			
US01	En tant qu’ utilisateur , je souhaite m’authentifier avec email et mot de passe afin d’accéder à mes informations de manière sécurisée.	• Développer l’API du Login • Mettre en place la gestion de session (JWT) • Réaliser l’interface connexion login • Consommer l’API d’authentification • Gérer les tentatives échouées (3 essais) avec verrouillage temporaire	2

US02	En tant qu' utilisateur , je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	• Créer les formulaires "mot de passe oublié" et de réinitialisation • Développer les API associées • Générer un token sécurisé avec expiration • Configurer l'envoi d'email de confirmation	2
US03	En tant qu' utilisateur , je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	• Intégrer les stratégies OAuth2 via Google et GitHub dans NestJS en utilisant Passport (@nestjs/passport) • Configurer les stratégies avec récupération et validation des données du profil utilisateur • Vérifier l'existence de l'utilisateur en base de données et une demande d'inscription OAuth est envoyée à l'admin • Implémenter les routes /auth/google, /auth/github et leurs callbacks, avec génération d'un JWT • Implémenter les boutons OAuth sur la page de login	3
US04	En tant qu' utilisateur , je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	• Développer l'endpoint POST /auth/logout avec révocation du refresh token en base • Supprimer les tokens et données utilisateur du localStorage • Réaliser le bouton de déconnexion	1
US05	En tant qu' utilisateur , je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	• Implémenter un système de rate limiting avec compteur de tentatives • Développer la logique de verrouillage temporaire après 3 tentatives échouées • Configurer l'envoi automatique d'email de notification via Nodemailer • Stocker les tentatives en base avec timestamp et IP	2

Epic 2 - Gestion des utilisateurs et profils

US06	En tant qu' utilisateur , je souhaite gérer mon profil afin que mes informations soient toujours à jour.	• Développer les endpoints GET /profil et PUT /profil pour consulter et modifier les informations • Implémenter l'upload de la photo de profil POST /profil/upload-photo • Développer le changement de mot de passe avec vérification de l'ancien • Réaliser l'interface profil côté Next.js avec formulaire prérempli	2
US07	En tant qu' administrateur , je souhaite créer des utilisateurs afin de gérer les accès au système.	• Développer l'endpoint POST /users pour créer un compte avec la génération automatique d'un mot de passe temporaire hashé via bcrypt • Valider les données d'entrée via class-validator • Réaliser le formulaire d'ajout côté Next.js • Envoyer un e-mail de bienvenue via Nodemailer avec les identifiants générés	3
US08	En tant qu' administrateur , je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	• Développer l'endpoint PUT /users/:id • Sécuriser l'accès à l'endpoint pour les administrateurs uniquement • Valider les données via class-validator • Réaliser le formulaire de modification	2
US09	En tant qu' administrateur , je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	• Développer les endpoints PATCH /users/:id/desactiver et PATCH /users/:id/reactiver • Implémenter les vérifications de réassignation des tickets et des demandes en cas de désactivation • Envoyer des notifications pour informer l'utilisateur de l'activation ou désactivation de son compte • Réaliser les boutons d'action avec appels des API	3

US10	En tant qu' administrateur , je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	• Développer l'endpoint GET /users retournant la liste des utilisateurs • Réaliser l'interface d'administration avec gestion	1
US11	En tant qu' utilisateur , je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	• Développer l'endpoint GET /users/:id/activities pour récupérer l'historique • Implémenter la logique de filtrage dans le backend en utilisant l'opérateur contains • Implémenter une barre de recherche dynamique • Réaliser l'interface timeline avec filtres par date et type d'action	2
Epic 3 - Gestion des rôles et permissions			
US12	En tant qu' administrateur , je souhaite gérer des rôles afin de structurer les accès au système.	• Développer les endpoints CRUD pour la gestion complète des rôles • Implémenter le PermissionsGuard et le décorateur @Permissions pour sécuriser les routes du backend • Réaliser l'interface de gestion sous forme de matrice	3
US13	En tant qu' administrateur , je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	• Développer l'endpoint PATCH /roles-permissions/roles/:id/sync pour synchroniser dynamiquement les associations entre un rôle et ses permissions. • Implémenter la logique de mise à jour atomique • Développer l'interface interactive de type "Matrice de contrôle d'accès"	2

US14	En tant qu' administrateur , je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	• Développer l'endpoint GET /roles-permissions/matrix) pour récupérer les données • Implémenter la logique de structuration des données au format JSON • Réaliser l'interface de visualisation de type "Heatmap" ou "Tableau de bord"	3
Epic 4 - Demandes d'inscription OAuth			
US15	En tant qu' utilisateur externe , je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	• Développer l'endpoint POST /auth/oauth/request pour créer une demande • Enregistrer la demande en base avec statut EN_ATTENTE • Implémenter l'envoi de notification à l'administrateur via email et in-app • Réaliser l'interface de demande d'accès	3
US16	En tant qu' administrateur , je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	• Développer les endpoints PATCH /auth/oauth/requests/:id/approve et /reject • Implémenter la création automatique du compte utilisateur lors de l'approbation • Configurer l'envoi d'emails de confirmation ou rejet via Nodemailer • Réaliser l'interface de gestion des demandes avec actions d'approbation/rejet	3
Epic 5 - Gestion Clients			

US17	En tant qu' administrateur , je souhaite créer un profil client afin de centraliser ses informations.	• Développer l'endpoint POST /clients pour orchestrer la création simultanée du compte CLIENT et du profil entreprise associé. • Implémenter un service de récupération automatique de logo (LogoService) établi sur le nom de la société • Intégrer une logique de gestion des projets historiques pour associer des projets passés, des chefs de projets et des membres d'équipe au nouveau client • Réaliser l'interface de formulaire	1
US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	• Développer l'endpoint GET /clients intégrant une logique de filtrage RBAC. • Réaliser une interface de visualisation	2
US19	En tant qu' administrateur , je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	• Développer l'endpoint GET /clients/ :id pour récupérer les informations du client. • Réaliser une interface de profil structurée. • Développer des composants spécifiques pour l'affichage des informations secondaires	1
US20	En tant qu' administrateur , je souhaite modifier les informations d'un client afin de maintenir des données à jour.	• Développer l'endpoint PATCH /clients/ :id permettant la mise à jour des informations des clients. • Développer des contrôles d'intégrité côté serveur pour valider les formats de données. • Réaliser une interface d'édition fluide.	2
US21	En tant qu' administrateur , je souhaite activer et désactiver un compte client afin de contrôler l'accès.	• Développer les endpoints de changement de statut (POST /desactiver, POST /reactiver) • Implémenter la vérification pour bloquer l'opération si le client possède des contrats actifs, des demandes en attente ou des tickets non résolus. • Intégrer le EmailService pour l'envoi automatique de notifications transactionnelles de changement de statut.	3

US22	En tant qu' administrateur , je souhaite ajouter des notes privées sur un client afin de conserver des informations internes.	• Développer un endpoint POST /clients/ :id/notes permettant d'ajouter des chaînes de caractères dans le tableau notesPrivées. • Réaliser une interface de type "Bloc-notes" ou "Journal interne" dans le profil client. • Sécuriser l'accès à ces informations au niveau du backend pour garantir que ces notes ne soient jamais exposées au client lui-même	2
US23	En tant qu' administrateur , je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	1
Epic 5 - Gestion des projets et équipes			
US24	En tant qu' administrateur , je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
US25	En tant qu' administrateur , je souhaite consulter la liste des projets afin de piloter l'activité.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	2
US26	En tant qu' administrateur , je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	3
US27	En tant que membre d'équipe , je souhaite consulter les informations du projet afin de comprendre le contexte.	• Développer l'endpoint POST /clients pour créer un client avec validation des données	1

II Analyse

Dans le processus de développement, l'analyse joue un rôle important ; elle permet d'identifier les exigences du système et de traduire les besoins exprimés en spécifications claires et cohérentes. C'est pour cela que nous allons présenter les diagrammes ainsi que les tableaux descriptifs des cas d'utilisation

1 Diagramme des cas d'utilisation du premier sprint

La figure ci-dessous présente le diagramme de cas d'utilisation du premier sprint :

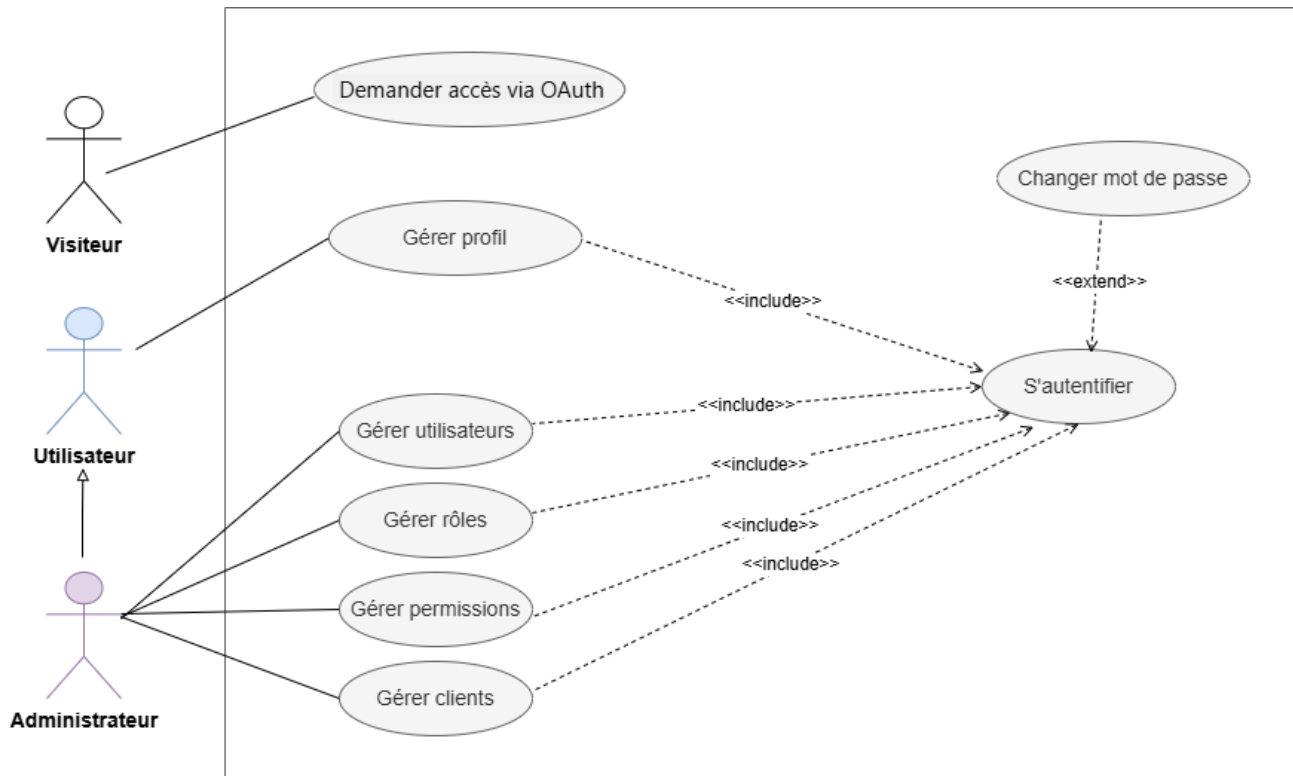


FIGURE III.1 – Diagramme des cas d'utilisation du premier sprint

Nous allons accompagner les cas d'utilisation d'un texte descriptif pour expliquer le scénario de chaque cas ainsi que ses exceptions.

2 Description textuelle de cas d'utilisation « Changer mot de passe »

La figure ci-dessous présente la description textuelle du cas d'utilisation « Changer le mot de passe » :

Nom du cas d'utilisation	Changer mot de passe
Acteurs	Utilisateurs
Préconditions	L'utilisateur possède déjà un compte
Postconditions	Le mot de passe est réinitialisé
Scénario principal	1. Le système affiche un formulaire de changement de mot de passe. 2. L'utilisateur saisit son e-mail. 3. L'utilisateur clique sur le bouton « Réinitialiser le mot de passe ». 4. Le système vérifie l'existence de l'email. 5. Le système envoie un e-mail à l'utilisateur. 6. L'utilisateur clique sur le lien reçu par e-mail. 7. Le système affiche le formulaire de réinitialisation de mot de passe. 8. L'utilisateur saisit son nouveau mot de passe deux fois pour confirmation. 9. Le système valide et enregistre le nouveau mot de passe. 10. Le système envoie un e-mail de confirmation à l'utilisateur.
Scénarios d'exception	• L'utilisateur saisit un e-mail qui ne correspond à aucun compte existant. • L'utilisateur a rempli un champ invalide, ou a laissé un champ vide. • Le lien de réinitialisation est expiré. • L'utilisateur ne confirme pas correctement le mot de passe.

TABLE III.2 – Description textuelle de cas d'utilisation « Changer mot de passe »

3 Description textuelle de cas d'utilisation « Demander accès via OAuth »

TABLE III.3 – Description textuelle « Demander accès via OAuth »

Nom du cas d'utilisation	Demander accès via OAuth
Acteurs	Visiteur
Préconditions	Le visiteur n'a pas de compte dans le système
Postconditions	Une demande d'inscription est créée et en attente de validation par un administrateur
Scénario principal	1. Le visiteur accède à la page de connexion. 2. Le visiteur sélectionne le rôle souhaité (Client, Développeur, Chef de projet). 3. Le visiteur clique sur "Se connecter avec Google" ou "Se connecter avec GitHub". 4. Le système redirige le visiteur vers le fournisseur OAuth sélectionné. 5. Le visiteur s'authentifie et autorise l'application à accéder à ses informations. 6. Le fournisseur OAuth redirige le visiteur vers l'application avec un code d'autorisation. 7. Le système échange le code d'autorisation contre un access token. 8. Le système récupère les informations du profil utilisateur (email, prénom, nom, photo). 9. Le système vérifie si un compte existe déjà avec cet email. 10. Si aucun compte n'existe : le système crée une nouvelle demande d'inscription en attente de validation. 11. Le système envoie un e-mail de notification à tous les administrateurs actifs. 12. Le système envoie un e-mail de confirmation au visiteur. 13. Le système redirige le visiteur vers la page de connexion avec un message "Demande en attente de validation".
Scénarios alternatifs	• Compte existant actif : le système connecte directement l'utilisateur. • Demande déjà soumise : le système affiche "Votre demande est déjà en attente" sans créer de doublon. • Annulation OAuth : le visiteur annule l'autorisation, le système redirige vers la page de connexion.
Scénarios d'exception	• Email non disponible : le fournisseur OAuth ne fournit pas l'email, le système affiche un message d'erreur. • Fournisseur OAuth indisponible : le système affiche un message d'erreur technique.

4 Analyse de la fonctionnalité « Gérer utilisateurs »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des utilisateurs afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

4.1 Raffinement de cas d'utilisation « Gérer utilisateurs »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des utilisateurs :

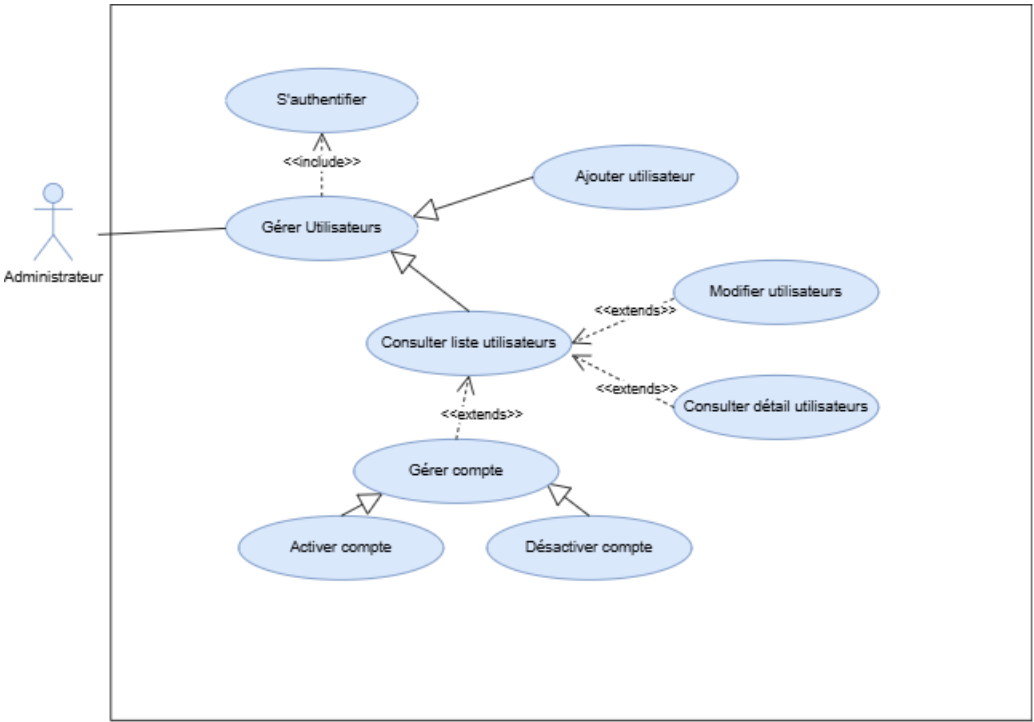


FIGURE III.2 – Diagramme des cas d’utilisation « Gérer utilisateurs »

4.2 Description textuelle de cas d’utilisation « Consulter liste utilisateurs »

TABLE III.4 – Description textuelle « Consulter liste utilisateurs »

Nom du cas d’utilisation	Consulter liste utilisateurs
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les utilisateurs
Postconditions	Liste des utilisateurs affichée
Scénario principal	1. L'administrateur accède à la section "Utilisateurs". 2. Le système affiche les utilisateurs sous forme de grille (par défaut) ou de liste selon le choix de l'utilisateur. 3. L'administrateur recherche un utilisateur en saisissant des informations clés dans la barre de recherche ou par des boutons de filtre (par statut, par rôle). 4. Le système filtre la liste.
Scénarios alternatifs	L'administrateur clique sur le bouton « Exporter ».

4.3 Description textuelle de cas d'utilisation « Ajouter utilisateur »

TABLE III.5 – Description textuelle « Ajouter utilisateur »

Nom du cas d'utilisation	Ajouter utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de créer des utilisateurs Liste des utilisateurs affichée
Postconditions	Utilisateur est ajouté
Scénario principal	1. L'administrateur clique sur le bouton "Nouvel Utilisateur". 2. Le système affiche le formulaire d'ajout. 3. L'administrateur remplit le formulaire. 4. L'administrateur valide le formulaire en cliquant sur (Créer le compte). 5. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur a rempli un champ invalide, ou a laissé un champ vide. • L'administrateur saisit un e-mail déjà utilisé dans un autre compte. • L'administrateur tente de fermer la fenêtre alors qu'il a commencé à saisir des données.
Scénarios alternatifs	Le système génère un mot de passe temporaire.

4.4 Description textuelle de cas d'utilisation « Modifier utilisateur »

TABLE III.6 – Description textuelle « Modifier utilisateur »

Nom du cas d'utilisation	Modifier utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de modifier les utilisateurs Liste des utilisateurs affichée
Postconditions	L'utilisateur est modifié
Scénario principal	1. L'administrateur consulte le profil de l'utilisateur choisi. 2. L'administrateur clique sur le bouton (Modifier). 3. Le système affiche le formulaire de modification. 4. L'administrateur modifie les informations souhaitées. 5. L'administrateur clique sur le bouton (Enregistrer). 6. Le système fait la mise à jour.
Scénarios alternatifs	L'administrateur supprime leur dernier rôle. L'utilisateur tente d'ajouter un rôle qui ne correspond pas au parcours métier.

4.5 Description textuelle de cas d'utilisation « Désactiver un utilisateur »

TABLE III.7 – Description textuelle « Désactiver un utilisateur »

Nom du cas d'utilisation	Désactiver un utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de désactiver les utilisateurs Liste des utilisateurs affichée L'utilisateur à désactiver doit être actuellement Actif
Postconditions	Le compte est désactivé
Scénario principal	1. L'administrateur choisit un compte dans la liste. 2. L'administrateur clique sur l'action (Désactiver). 3. Le système vérifie si l'utilisateur possède des tickets actifs ou des demandes en attente. 4. Le système affiche une boîte de dialogue de confirmation. 5. L'administrateur confirme l'action. 6. Le système met à jour le statut de l'utilisateur. 7. Le système envoie un e-mail automatique à l'utilisateur pour l'informer de la suspension de son compte.
Scénarios alternatifs	L'administrateur réassigne les tickets ou les demandes.
Scénarios d'exception	• L'utilisateur tente de désactiver un utilisateur qui possède des tickets actifs ou des demandes en attente.

5 Analyse de la fonctionnalité « Gérer profil »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des profils afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

5.1 Raffinement de cas d'utilisation « Gérer Profil »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion de profil :

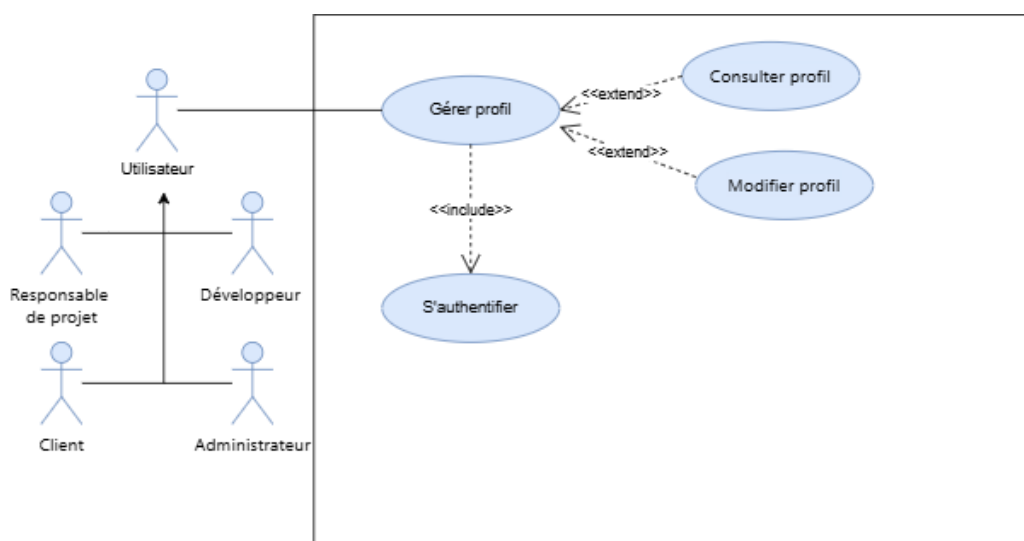


FIGURE III.3 – Raffinement de cas d'utilisation « Gérer Profil »

5.2 Description textuelle de cas d'utilisation « Consulter Profil »

TABLE III.8 – Description textuelle « Consulter profil »

Nom du cas d'utilisation	Consulter profil
Acteurs	Utilisateur
Préconditions	L'utilisateur est authentifié
Postconditions	Le profil choisi est consulté
Scénario principal pour consulter son propre profil	1. L'utilisateur clique sur (Voir le profil). 2. Le système affiche la page personnelle. 3. L'utilisateur consulte ses données et son historique de modification.
Consulter le profil d'un utilisateur	1. L'administrateur choisit un utilisateur dans la liste puis clique sur (Voir profil). 2. Le système affiche les informations de l'employé.

5.3 Description textuelle de cas d'utilisation « Modifier Profil »

TABLE III.9 – Description textuelle « Modifier le Profil »

Nom du cas d'utilisation	Modifier le Profil
Acteurs	Administrateur, Utilisateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de modifier son propre profil ou celui d'autres utilisateurs (selon le rôle)
Postconditions	Le Profil est modifié
Scénario principal pour Modifier le profil d'un utilisateur	1. L'administrateur accède au profil de l'utilisateur. 2. L'administrateur modifie les accès en ajoutant et retirant des rôles système. 3. L'administrateur change le statut du compte (désactiver ou activer). 4. Le système valide chaque action.
Scénario principal pour Modifier son propre profil	1. L'utilisateur accède à sa page "Mon profil". 2. L'utilisateur met à jour sa photo de profil ou ses coordonnées. 3. L'utilisateur change son mot de passe en saisissant son ancien mot de passe pour validation. 4. L'utilisateur ajoute ou définit un nouvel e-mail et choisit lequel serait le principal. 5. Le système enregistre les changements.
Scénarios d'exception	• L'administrateur veut retirer le dernier rôle. • L'utilisateur veut modifier ses propres rôles.

6 Analyse de la fonctionnalité « Gérer rôles »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des rôles afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

6.1 Raffinement de cas d'utilisation « Gérer rôles »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des rôles :

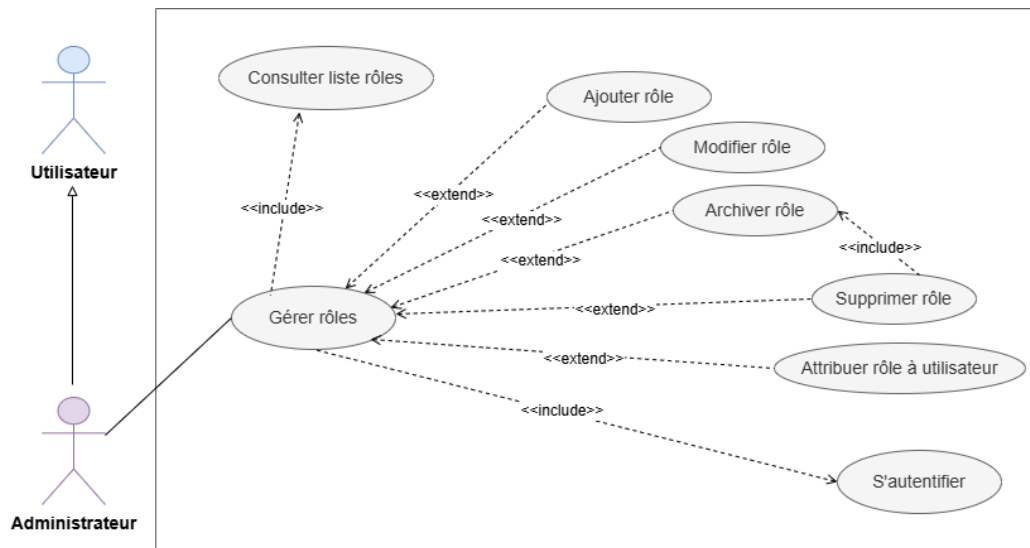


FIGURE III.4 – Raffinement de cas d'utilisation « Gérer rôles »

6.2 Description textuelle de cas d'utilisation « Consulter liste rôles »

TABLE III.10 – Description textuelle « Consulter liste rôles »

Nom du cas d'utilisation	Consulter liste rôles
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les rôles
Postconditions	Liste des rôles affichée
Scénario principal	1. L'administrateur accède à la section "Gestion Rôles". 2. Le système affiche la liste des rôles avec leurs descriptions et le nombre d'utilisateurs assignés. 3. L'administrateur recherche un rôle en saisissant des informations clés dans la barre de recherche. 4. Le système filtre la liste. 5. L'administrateur peut consulter les permissions associées à chaque rôle.
Scénarios alternatifs	L'administrateur clique sur un rôle pour voir les utilisateurs qui le possèdent.

6.3 Description textuelle de cas d'utilisation « Ajouter rôle »

TABLE III.11 – Description textuelle « Ajouter rôle »

Nom du cas d'utilisation	Ajouter rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de créer des rôles Liste des rôles affichée
Postconditions	Rôle est ajouté
Scénario principal	1. L'administrateur clique sur le bouton "Nouveau Rôle". 2. Le système affiche le formulaire d'ajout. 3. L'administrateur saisit le nom du rôle et sa description. 4. L'administrateur sélectionne les permissions à associer au rôle. 5. L'administrateur valide le formulaire en cliquant sur (Créer le rôle). 6. Le système enregistre le nouveau rôle. 7. Le système attribue automatiquement les permissions de base. 8. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur a rempli un champ invalide, ou a laissé un champ obligatoire vide. • L'administrateur saisit un nom de rôle déjà existant.

6.4 Description textuelle de cas d'utilisation « Modifier rôle »

TABLE III.12 – Description textuelle « Modifier rôle »

Nom du cas d'utilisation	Modifier rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de modifier les rôles Liste des rôles affichée
Postconditions	Le rôle est modifié
Scénario principal	1. L'administrateur sélectionne le rôle à modifier. 2. L'administrateur clique sur le bouton (Modifier). 3. Le système affiche le formulaire de modification avec les informations actuelles. 4. L'administrateur modifie le nom, la description ou les permissions associées. 5. L'administrateur clique sur le bouton (Enregistrer). 6. Le système met à jour le rôle. 7. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur tente de retirer toutes les permissions d'un rôle. • L'administrateur modifie un rôle système protégé.

6.5 Description textuelle de cas d'utilisation « Archiver rôle »

TABLE III.13 – Description textuelle « Archiver rôle »

Nom du cas d'utilisation	Archiver rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'archiver les rôles Le rôle ne doit pas être assigné à des utilisateurs
Postconditions	Le rôle est archivé
Scénario principal	1. L'administrateur sélectionne un rôle dans la liste. 2. L'administrateur clique sur l'action (Archiver). 3. Le système vérifie que le rôle n'est pas assigné à des utilisateurs. 4. Le système affiche une confirmation. 5. L'administrateur confirme l'action. 6. Le système marque le rôle comme archivé. 7. Le système affiche le message de succès.
Scénarios d'exception	Le rôle est encore assigné à des utilisateurs.
Scénarios alternatifs	L'administrateur restaure un rôle archivé.

6.6 Description textuelle de cas d'utilisation « Supprimer rôle »

TABLE III.14 – Description textuelle « Supprimer rôle »

Nom du cas d'utilisation	Supprimer rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de supprimer les rôles Liste des rôles affichée Le rôle doit être archivé Le rôle ne doit pas être un rôle système protégé
Postconditions	Le rôle est supprimé définitivement
Scénario principal	1. L'administrateur filtre la liste pour afficher les rôles archivés. 2. L'administrateur sélectionne un rôle archivé dans la liste. 3. L'administrateur clique sur l'action (Supprimer définitivement). 4. Le système affiche une boîte de dialogue de confirmation. 5. L'administrateur confirme l'action. 6. Le système supprime le rôle et toutes ses liaisons de la base de données. 7. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur restaure le rôle archivé au lieu de le supprimer (action "Restaurer").

6.7 Description textuelle de cas d'utilisation « Attribuer rôle à utilisateur »

TABLE III.15 – Description textuelle « Attribuer rôle à utilisateur »

Nom du cas d'utilisation	Attribuer rôle à utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'attribuer des rôles Utilisateur et rôle existent
Postconditions	Le rôle est attribué à l'utilisateur
Scénario principal	1. L'administrateur consulte le profil d'un utilisateur. 2. L'administrateur clique sur "Ajouter un rôle". 3. Le système affiche la liste des rôles disponibles. 4. L'administrateur sélectionne un rôle. 5. L'administrateur valide la sélection. 6. Le système vérifie les règles d'exclusivité (CLIENT ne peut pas avoir CHEF_PROJET/DEVELOPPEUR/ADMIN). 7. Le système crée l'attribution. 8. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur peut aussi attribuer le rôle depuis les détails du rôle en sélectionnant un ou plusieurs utilisateurs à qui l'attribuer.
Scénarios d'exception	• Le rôle est déjà attribué à cet utilisateur. • Violation des règles d'exclusivité des rôles.

6.8 Description textuelle de cas d'utilisation « Retirer rôle d'un utilisateur »

TABLE III.16 – Description textuelle « Retirer rôle utilisateur »

Nom du cas d'utilisation	Retirer rôle utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de retirer des rôles Le rôle est attribué à l'utilisateur
Postconditions	Le rôle est retiré de l'utilisateur
Scénario principal	1. L'administrateur consulte le profil d'un utilisateur. 2. L'administrateur visualise les rôles actuels de l'utilisateur. 3. L'administrateur clique sur l'icône de suppression d'un rôle. 4. Le système affiche une confirmation. 5. L'administrateur confirme l'action. 6. Le système retire le rôle. 7. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur tente de retirer le dernier rôle de l'utilisateur. • Le rôle n'est pas attribué à cet utilisateur.

7 Description des cas d'utilisation « Gérer permissions »

7.1 Raffinement de cas d'utilisation « Gérer permissions »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des permissions :

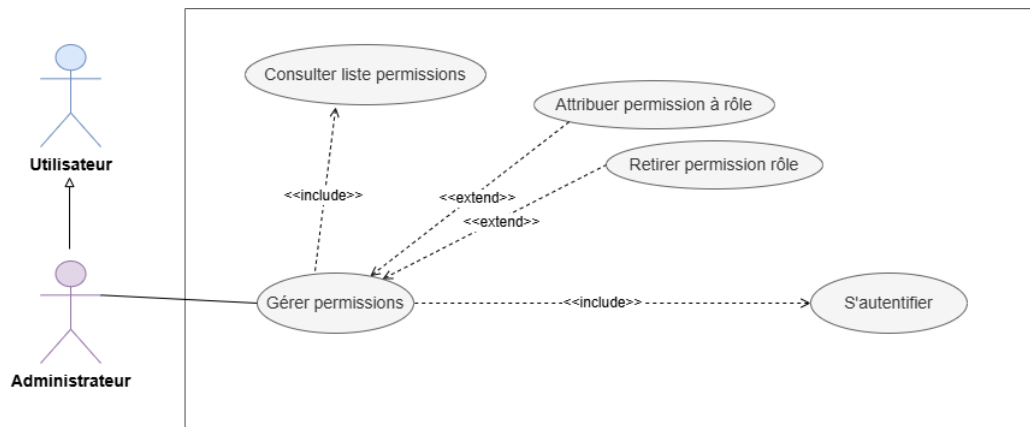


FIGURE III.5 – Raffinement de cas d'utilisation « Gérer permissions »

7.2 Description textuelle de cas d'utilisation « Consulter liste des permissions »

TABLE III.17 – Description textuelle « Consulter liste des permissions »

Nom du cas d'utilisation	Consulter liste des permissions
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les permissions
Postconditions	Liste des permissions affichée
Scénario principal	1. L'administrateur accède à la section "Permissions". 2. Le système affiche une matrice avec les rôles en horizontal et les catégories de permissions en vertical. 3. L'administrateur peut visualiser quelles permissions sont attribuées à chaque rôle (cases cochées). 4. L'administrateur peut filtrer par catégorie de permissions ou par rôle.
Scénarios alternatifs	• L'administrateur sélectionne la vue "Par rôle" pour afficher une liste des rôles avec leurs permissions respectives organisées par catégorie. • L'administrateur clique sur le bouton « Exporter » pour télécharger la matrice des permissions.

7.3 Description textuelle de cas d'utilisation « Attribuer permission à rôle »

TABLE III.18 – Description textuelle « Attribuer permission à rôle »

Nom du cas d'utilisation	Attribuer permission à rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'attribuer des permissions aux rôles Rôle et permission existent
Postconditions	La permission est ajoutée au rôle
Scénario principal	1. L'administrateur consulte un rôle spécifique. 2. L'administrateur clique sur l'onglet "Permissions". 3. Le système affiche toutes les permissions organisées par catégorie avec des cases à cocher. 4. L'administrateur coche les permissions à attribuer. 5. L'administrateur clique sur "Enregistrer". 6. Le système crée les liaisons. 7. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur utilise la matrice des permissions pour cocher les cases correspondant aux permissions à attribuer pour plusieurs rôles simultanément.
Scénarios d'exception	La permission est déjà attribuée au rôle.

7.4 Description textuelle de cas d'utilisation « Retirer permission d'un rôle »

TABLE III.19 – Description textuelle « Retirer permission rôle »

Nom du cas d'utilisation	Retirer permission rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de retirer des permissions aux rôles La permission est attribuée au rôle
Postconditions	La permission est retirée du rôle
Scénario principal	1. L'administrateur consulte un rôle spécifique. 2. L'administrateur clique sur l'onglet "Permissions". 3. Le système affiche toutes les permissions organisées par catégorie avec des cases à cocher. 4. L'administrateur décoche les permissions à retirer. 5. L'administrateur clique sur "Enregistrer". 6. Le système retire les permissions décochées. 7. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur utilise la matrice des permissions pour décocher les cases correspondant aux permissions à retirer pour plusieurs rôles simultanément.
Scénarios d'exception	La permission n'est pas attribuée à ce rôle.

8 Description des cas d'utilisation « Gérer clients »

8.1 Raffinement de cas d'utilisation « Gérer clients »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des clients :

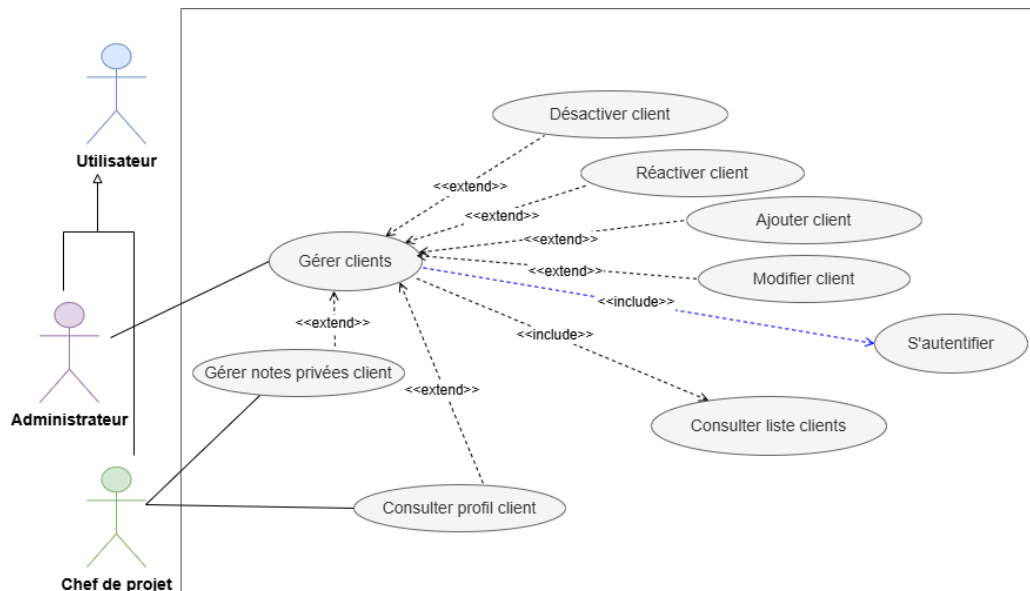


FIGURE III.6 – Raffinement de cas d'utilisation « Gérer clients »

8.2 Description textuelle de cas d'utilisation « Consulter liste clients »

TABLE III.20 – Description textuelle « Consulter liste clients »

Nom du cas d'utilisation	Consulter liste clients
Acteurs	Administrateur, Chef de projet
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les clients
Postconditions	Liste des clients affichée
Scénario principal	1. L'utilisateur accède à la section "Clients". 2. Le système affiche les clients sous forme de grille (par défaut). 3. L'utilisateur peut basculer entre trois modes d'affichage : grille, liste, ou vue par société. 4. L'utilisateur recherche un client en saisissant des informations clés dans la barre de recherche ou par des boutons de filtre (par statut, par secteur d'activité). 5. Le système filtre les résultats.
Scénarios alternatifs	• L'utilisateur sélectionne la vue "Par société" pour regrouper les clients par société et visualiser les contacts associés à chaque société. • L'utilisateur clique sur le bouton « Exporter » pour télécharger la liste des clients (CSV, Excel). • L'utilisateur clique sur le bouton « Importer » pour ajouter plusieurs clients via un fichier (CSV, Excel).

8.3 Description textuelle de cas d'utilisation « Ajouter client »

TABLE III.21 – Description textuelle « Ajouter client »

Nom du cas d'utilisation	Ajouter client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de créer des clients Liste des clients affichée
Postconditions	Client est ajouté
Scénario principal	1. L'administrateur clique sur le bouton "Nouveau Client". 2. Le système affiche le formulaire d'ajout. 3. L'administrateur remplit les informations personnelles du client (prénom, nom, e-mail, mot de passe). 4. L'administrateur sélectionne une société existante dans la liste déroulante ou crée une nouvelle société. 5. Si nouvelle société : l'administrateur remplit les informations de la société (nom, adresse, secteur d'activité, téléphone, site web). 6. Le système récupère automatiquement le logo de la société via une API externe en utilisant le nom de la société ou le domaine du site web. 7. L'administrateur valide le formulaire en cliquant sur (Créer le client). 8. Le système crée l'utilisateur et son profil client associé à la société. 9. Le système envoie un e-mail de bienvenue avec les identifiants. 10. Le système affiche le message de succès.
Scénarios alternatifs	• L'administrateur télécharge manuellement le logo de la société au lieu d'utiliser la récupération automatique.
Scénarios d'exception	• L'administrateur a rempli un champ invalide, ou a laissé un champ obligatoire vide. • L'administrateur saisit un e-mail déjà utilisé pour un autre utilisateur. • L'administrateur tente de fermer la fenêtre alors qu'il a commencé à saisir des données.

8.4 Description textuelle de cas d'utilisation « Modifier client »

TABLE III.22 – Description textuelle « Modifier client »

Nom du cas d'utilisation	Modifier client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de modifier les clients Liste des clients affichée
Postconditions	Le client est modifié
Scénario principal	1. L'administrateur consulte la fiche du client choisi. 2. L'administrateur clique sur le bouton (Modifier). 3. Le système affiche le formulaire de modification. 4. L'administrateur modifie les informations souhaitées. 5. L'administrateur clique sur le bouton (Enregistrer). 6. Le système met à jour les informations du client. 7. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur saisit un e-mail déjà utilisé par un autre utilisateur. • L'administrateur tente de modifier un client ayant des contrats actifs sans validation.

8.5 Description textuelle de cas d'utilisation « Consulter profil client »

TABLE III.23 – Description textuelle « Consulter profil client »

Nom du cas d'utilisation	Consulter profil client
Acteurs	Administrateur, Chef de projet, Client (son propre profil)
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les profils clients ou consulte son propre profil Pour Chef de projet : le client doit être associé à au moins un de ses projets
Postconditions	Profil client consulté
Scénario principal	1. L'utilisateur accède au profil du client. 2. Le système affiche les informations personnelles et société. 3. Le système affiche les statistiques et éléments accessibles selon le rôle de l'utilisateur. 4. L'utilisateur peut naviguer vers les éléments disponibles (projets, demandes, tickets, documents).
Restrictions d'accès	• Administrateur : accès complet (contrats, projets, demandes, tickets, notes privées, documents). • Chef de projet : accès limité aux projets où il est assigné et aux demandes/tickets liés à ces projets uniquement. • Client : accès à son propre profil, ses projets, demandes et tickets.
Scénarios alternatifs	• L'utilisateur exporte les informations du profil en PDF. • L'utilisateur clique sur "Contacter" pour envoyer un e-mail ou un message via la messagerie interne de l'application. • L'utilisateur clique sur l'adresse de la société pour visualiser sa localisation sur une carte interactive. • L'administrateur ajoute une note privée sur le client.

8.6 Description textuelle de cas d'utilisation « Désactiver client »

TABLE III.24 – Description textuelle « Désactiver un client »

Nom du cas d'utilisation	Désactiver un client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de désactiver les clients Liste des clients affichée Le client à désactiver doit être actuellement Actif
Postconditions	Le compte client est désactivé
Scénario principal	1. L'administrateur choisit un client dans la liste. 2. L'administrateur clique sur l'action (Désactiver). 3. Le système vérifie si le client possède des contrats actifs, des demandes en cours ou des tickets actifs. 4. Le système affiche une boîte de dialogue de confirmation avec les détails des contrats, demandes et tickets actifs. 5. L'administrateur confirme l'action. 6. Le système met à jour le statut du client. 7. Le système révoque les sessions actives (suppression des refresh tokens). 8. Le système envoie un e-mail automatique au contact principal du client pour l'informer de la désactivation. 9. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur tente de désactiver un client qui possède des contrats actifs sans les clôturer. • L'administrateur tente de désactiver un client ayant des demandes en attente de traitement. • L'administrateur tente de désactiver un client ayant des tickets actifs non résolus.
Scénarios alternatifs	L'administrateur clôture les contrats, traite les demandes et résout les tickets avant la désactivation.

8.7 Description textuelle de cas d'utilisation « Réactiver client »

TABLE III.25 – Description textuelle « Réactiver un client »

Nom du cas d'utilisation	Réactiver un client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de réactiver les clients Le compte client doit être désactivé
Postconditions	Le compte client est réactivé
Scénario principal	1. L'administrateur consulte la liste des clients désactivés. 2. L'administrateur sélectionne un client à réactiver. 3. L'administrateur clique sur l'action (Réactiver le compte). 4. Le système affiche une confirmation. 5. L'administrateur confirme l'action. 6. Le système réactive le compte. 7. Le système envoie un e-mail au client. 8. Le système affiche le message de succès.
Scénarios d'exception	Le client est déjà actif.

8.8 Description textuelle de cas d'utilisation « Gérer notes privées client »

TABLE III.26 – Description textuelle « Gérer notes privées client »

Nom du cas d'utilisation	Gérer notes privées client
Acteurs	Administrateur, Chef de projet
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de gérer les notes privées des clients Profil client consulté
Postconditions	Notes privées mises à jour
Scénario principal	1. L'utilisateur accède au profil du client. 2. L'utilisateur consulte la section "Notes privées". 3. Pour ajouter une note : l'utilisateur clique sur "Ajouter", saisit le contenu et valide. 4. Pour supprimer une note : l'utilisateur clique sur l'icône de suppression et confirme. 5. Le système enregistre les modifications. 6. Le système affiche le message de succès.
Scénarios alternatifs	Les membres de l'équipe (Développeur, Technicien) peuvent consulter (lecture seule) les notes privées du client dans la section détails du ticket pour avoir le contexte nécessaire au traitement.
Scénarios d'exception	L'utilisateur tente d'ajouter une note vide.

III Diagramme de classes de première sprint

IV Schéma relationnel

V Réalisation

Dans cette partie, nous présentons les interfaces développées au cours de ce sprint.

1 Diagramme de compositions

2 Description des interfaces

VI Tests et Validation

La phase de test est essentielle pour garantir la qualité et la fiabilité de la solution. Elle valide que chaque fonctionnalité répond aux exigences métier et identifie les anomalies avant la mise en production. Dans ce sprint, nous avons testé les mécanismes d'authentification, la gestion des utilisateurs et le contrôle d'accès (RBAC) pour assurer la robustesse de nos implémentations.

1 Stratégie de tests adoptée

Dans le cadre de ce projet, nous avons choisi une stratégie de tests structurée, notre objectif est de vérifier le bon fonctionnement de l'application à plusieurs niveaux, tout en gardant un équilibre entre la couverture des fonctionnalités et le temps d'exécution des tests.

La pyramide de tests se compose de trois niveaux :

1.1 Tests unitaires

Nous avons réalisé des tests unitaires via Vitest pour tester les règles métier et les validations de données. Ces tests s'exécutent rapidement, ce qui permet de détecter les erreurs dès le développement.

□ Test de changer mot de passe

Par ces tests, nous visons à valider le bon fonctionnement des méthodes **demanderReinitialisationMotDePasse()** et **reinitialiserMotDePasse()**, qui gèrent le processus de réinitialisation du mot de passe utilisateur.

En utilisant Vitest, nous avons isolé ces tests du reste de l'application afin d'assurer leur indépendance. L'objectif est de s'assurer que ces méthodes traitent correctement les cas d'erreur liés à la sécurité du processus de réinitialisation.

Nous avons couvert les deux cas de test suivants :

- Cas où l'e-mail fourni n'existe pas dans la base de données.
- Cas où le token de réinitialisation est invalide car il a été généré depuis plus d'une heure.

```
PASS src/modules/auth/auth.service.spec.ts
AuthService - Réinitialisation Mot de Passe
  demanderReinitialisationMotDePasse
    ✓ devrait lever NotFoundException si l'email n'existe pas en base (75 ms)
  reinitialiserMotDePasse
    ✓ devrait lever BadRequestException si le token a expiré depuis plus d'1 heure (43 ms)

Test Suites: 1 passed, 1 total
Tests:      2 passed, 2 total
Test Suites: 1 passed, 1 total
Test Suites: 1 passed, 1 total
Tests:      2 passed, 2 total
Snapshots:  0 total
Time:       3.342 s, estimated 26 s
```

FIGURE III.7 – Test unitaire de changer mot de passe

1.2 Tests d'intégration manuels de l'API

Par ce test, nous visons à valider le bon fonctionnement de la méthode **modifierProfil()** exposée via l'endpoint **PUT /api/profil**, qui permet à un utilisateur de mettre à jour les informations de son profil.

Nous avons couvert le cas de test suivant :

- Cas où l'e-mail fourni dans la requête ne respecte pas le format attendu.



FIGURE III.8 – Test de mise à jour du profil

1.3 Tests End-to-End

Par ces tests, nous visons à valider le bon fonctionnement du processus de désactivation et réactivation d'utilisateurs. En utilisant Playwright, nous avons testé les parcours complets de gestion du statut utilisateur en simulant les actions réelles d'un administrateur. L'objectif est de s'assurer que le système gère correctement les changements de statut et applique les restrictions d'accès appropriées. Nous avons couvert le cas de test suivant :

- Cas où l'administrateur tente de désactiver un utilisateur actif disposant de tickets/demandes en cours : le système détecte les 9 tickets actifs, affiche le modal de réassignation, procède à l'assignation de chaque ticket au premier développeur/chef de projet disponible, puis finalise la désactivation du compte après confirmation via le bouton « Réassigner et Désactiver ».

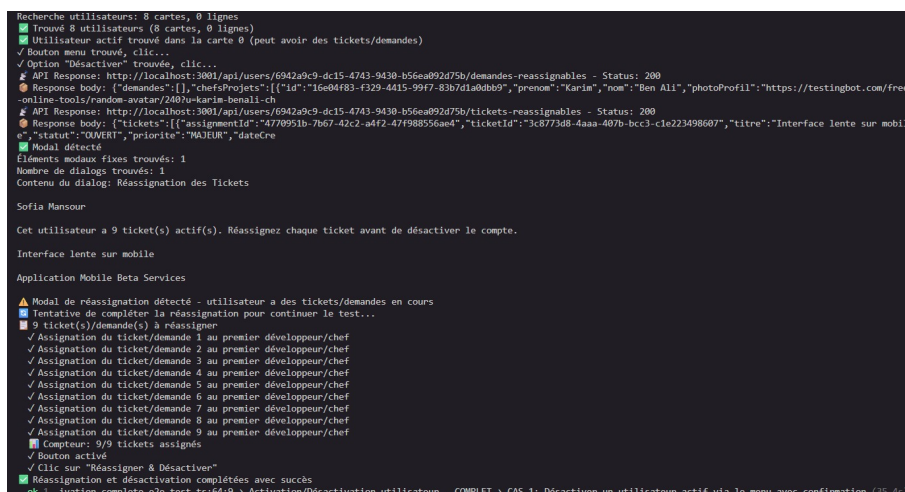


FIGURE III.9 – Test End-to-End de la gestion des utilisateurs via Playwright

CHAPITRE 4

Réalisation



Sommaire

4

I	Analyse	93
II	Conception	94
III	Réalisation	95
IV	Tests	96

Introduction

Dans ce chapitre, nous abordons le deuxième sprint du projet **SolidMaint**, couvrant la période du 12 mars au 2 avril 2026, soit trois semaines de développement consacrées aux modules de gestion métier essentiels de l’application. Nous commencerons par la présentation du backlog du sprint regroupant vingt-trois user stories réparties en quatre epics, portant sur la gestion des clients, des projets et des contrats de maintenance avec leurs SLA associés, suivie de la phase d’analyse et de conception de l’architecture et du modèle de données, puis de la réalisation des interfaces utilisateur, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l’effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 14 - Historique et traçabilité			
US95	En tant qu’ administrateur , je souhaite consulter l’historique complet de toutes les actions afin d’assurer la traçabilité.		2
US96	En tant qu’ administrateur , je souhaite exporter les logs d’audit afin de répondre aux exigences de conformité.		2
Epic 7 - Gestion des contrats			
US28	En tant qu’ administrateur , je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	• Développer l’API de création de contrat de maintenance • Mettre en place la gestion des paramètres du contrat (association client, dates, type SLA, cycle de facturation). • Réaliser l’interface du formulaire complet de création de contrat • Configurer le système de notification et d’alertes	3

ID US	User Story	Description technique	Effort (j)
US29	En tant qu' administrateur , je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	vspace2mm • Mise en place d'une API REST de récupération des contrats avec recherche • développement d'une interface tableau pour l'affichage • intégration côté frontend • implémentation d'une fonctionnalité d'exportation des données.	2
US30	En tant qu' administrateur , je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements.	• Mise en place de tâches planifiées (cron jobs) pour le calcul dynamique des échéances (J-30, J-15, J-7) • développement d'un service d'envoi de notifications multi-canal • réalisation de l'interface dédiée à leur affichage.	3
US31	En tant qu' administrateur , je souhaite archiver un contrat afin de conserver l'historique.	• Développement d'une API d'archivage et de restauration des contrats • réalisation d'une interface avec modales de confirmation.	1
US32	En tant que client , je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	• Développement d'une API de soumission des demandes de résiliation. • développement d'une interface formulaire. • intégration des appels API.	1

ID US	User Story	Description technique	Effort (j)
US33	En tant qu' administrateur , je souhaite résilier un contrat afin de l'interrompre avant terme.	• Implémentation d'une API de résiliation administrative • développement d'une interface dédiée côté administration • intégration de l'API et configuration de l'envoi automatique d'un avis de résiliation au client	2
US34	En tant qu' administrateur , je souhaite renouveler un contrat afin de prolonger la relation.		2
US35	En tant que chef de projet , je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration.	• Développement d'une API de récupération des échéances contractuelles • Réalisation d'une interface de vue calendrier • Implémentation d'une fonctionnalité d'exportation des données	2
US36	En tant qu' administrateur , je souhaite centraliser les documents d'un contrat afin de les retrouver facilement.	• Développement d'une API de récupération des échéances contractuelles • Implémentation d'un mécanisme de versioning automatique lors de la modification des fichiers • Réalisation d'une interface de gestion des pièces jointes • Intégration de l'API de liaison des documents	2

ID US	User Story	Description technique	Effort (j)
US37	En tant qu' administrateur , je souhaite configurer les règles SLA afin de garantir le respect des délais d'intervention et assurer une qualité de service optimale.	• Développement d'une API de gestion des politiques SLA • Réalisation d'une interface de paramétrage • Intégration de l'API de configuration des règles SLA • Implémentation de mécanismes d'alertes automatiques	2
Epic 14 - Historique et traçabilité			
US95	En tant qu' administrateur , je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.		2
US96	En tant qu' administrateur , je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.		2
Total Sprint 2 :			42 j

I Analyse

1 Diagramme des cas d'utilisation du Sprint 2

Cette section présentera le diagramme de cas d'utilisation du deuxième sprint, couvrant la gestion des clients, projets et contrats.

[À compléter avec le diagramme de cas d'utilisation]

2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 2.

[À compléter avec les descriptions textuelles]

II Conception

1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la gestion des clients, projets et contrats.

[À compléter avec les diagrammes de séquence]

2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des clients, projets, contrats et audit.

[À compléter avec le schéma de base de données]

III Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 2, incluant les modules de gestion des clients, projets et contrats.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 2.

[À compléter avec les captures d'écran]

IV Tests

1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 2.

[À compléter avec les résultats des tests]

2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

CHAPITRE 5

Chapitre 5



Sommaire

5

I	Analyse	102
II	Conception	103
III	Réalisation	104
IV	Tests	105

Introduction

Dans ce chapitre, nous abordons le troisième sprint du projet **SolidMaint**, couvrant la période du 3 avril au 1er mai 2026, soit quatre semaines de développement consacrées au cœur fonctionnel de l’application, à savoir la gestion des demandes clients et des tickets de maintenance. Nous commencerons par la présentation du backlog du sprint regroupant trente-six user stories, suivie de la phase d’analyse et de conception, puis de la réalisation des interfaces intégrant un tableau Kanban, un suivi du temps de travail, un système de gestion des SLA et des dashboards adaptés à chaque rôle, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l’effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 8 - Gestion des demandes			
US39	En tant que client , je souhaite soumettre une demande de maintenance afin de signaler un besoin.	Formulaire de soumission, upload de pièces jointes (max 5), génération numéro unique, confirmation email	2
US40	En tant que chef de projet , je souhaite consulter les demandes en attente afin de les traiter.	Liste filtrée par statut EN_ATTENTE, tri par date, indicateurs de priorité	1
US41	En tant que chef de projet , je souhaite valider une demande afin de la convertir en ticket.	Interface de validation, définition catégorie/priorité, création automatique de ticket lié	2
US42	En tant que chef de projet , je souhaite demander des informations complémentaires afin de clarifier une demande.	Formulaire de questions, changement statut INFOS_REQUISES, relance automatique après 48h	2
US43	En tant que client , je souhaite compléter ma demande afin de fournir les informations manquantes.	Formulaire de réponse, retour automatique au statut EN_ATTENTE	1

ID US	User Story	Description technique	Effort (j)
US44	En tant que chef de projet , je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Modification type/priorité avec justification obligatoire, traçabilité	1
US45	En tant qu' utilisateur , je souhaite commenter une demande afin de communiquer sur le contexte.	Éditeur riche, mentions @utilisateur, pièces jointes possibles	2
US46	En tant qu' utilisateur , je souhaite consulter l'historique d'une demande afin de suivre son évolution.	Timeline chronologique avec tous les événements, filtres par type, export PDF	1
Epic 9 - Gestion des tickets			
US47	En tant que développeur , je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Tableau Kanban avec colonnes (OUVERT, EN_COURS, EN_REVUE, RESOLU, FERME), drag & drop, indicateurs SLA	3
US48	En tant que chef de projet , je souhaite assigner un ticket à un développeur afin de répartir la charge.	Sélection développeur, vérification disponibilité/compétences, alerte surcharge	2
US49	En tant que chef de projet , je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Multi-sélection, définition rôles (principal/support), répartition temps estimé	2
US50	En tant que développeur , je souhaite me désassigner d'un ticket afin de libérer ma charge.	Désassignation avec motif obligatoire, retour statut OUVERT, notification chef projet	1
US51	En tant que développeur , je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Bouton Démarrer, création session de travail, indicateur visuel chrono actif	2
US52	En tant que développeur , je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Bouton Pause, enregistrement temps écoulé, possibilité de reprise	1

ID US	User Story	Description technique	Effort (j)
US53	En tant que développeur , je souhaite arrêter le chronomètre afin de finaliser ma session.	Arrêt avec commentaire optionnel, ajout session à l'historique, notification si dépassement	2
US54	En tant que développeur , je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Formulaire (date, heures, durée, commentaire), validation sans chevauchement	1
US55	En tant que développeur , je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Workflow avec transitions validées, notification parties prenantes	2
US56	En tant que client , je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Interface de validation avec commentaire, fermeture ticket, notification développeur	1
US57	En tant que chef de projet , je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Réouverture avec motif, réinitialisation SLA, notification assignés	1
US58	En tant que chef de projet , je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Affichage temps total et détaillé par développeur, graphiques	2
US59	En tant que développeur , je souhaite signaler un blocage afin d'alerter sur un obstacle.	Formulaire (type, description, impact), notification urgence chef projet, pause SLA	2
US60	En tant que membre d'équipe , je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Résolution avec commentaire obligatoire, reprise chrono SLA, traçabilité	1
US61	En tant que développeur , je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	Formulaire de demande (durée, justification), soumission pour validation	1
US62	En tant que chef de projet , je souhaite approuver ou rejeter les demandes d'extension SLA.	Liste demandes en attente, approbation/rejet avec commentaire, mise à jour auto SLA	1

ID US	User Story	Description technique	Effort (j)
US63	En tant qu' utilisateur , je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Éditeur riche, autocomplétion @utilisateur, distinction internes/externes	2
US64	En tant qu' utilisateur , je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Upload par drag & drop (images/PDF/logs, max 10MB), prévisualisation	1
US65	En tant que développeur , je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	WebSocket temps réel, toast notification, indicateur "X est en train d'éditer"	2
US66	En tant qu' utilisateur , je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	Timeline chronologique inversée, filtres par type, export PDF	1
Epic 13 - Dashboards			
US89	En tant qu' administrateur , je souhaite consulter un dashboard global afin de piloter le système.	KPIs (utilisateurs actifs, tickets ouverts, taux SLA, CA), graphiques, alertes, filtres période	3
US90	En tant que chef de projet , je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	Vue équipe (charge travail par dev, performance SLA, temps passé vs estimé)	3
US91	En tant que développeur , je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Tickets assignés avec priorités, chronomètre actif, temps passé, échéances SLA	2
US92	En tant que client , je souhaite consulter mon dashboard afin de suivre mes projets.	Contrats actifs, demandes/tickets en cours, statistiques, documents récents, contacts équipe	2
Total Sprint 3 :			54 j

I Analyse

1 Diagramme des cas d'utilisation du Sprint 3

Cette section présentera le diagramme de cas d'utilisation du troisième sprint, couvrant la gestion des demandes, tickets et dashboards.

[À compléter avec le diagramme de cas d'utilisation]

2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 3.

[À compléter avec les descriptions textuelles]

II Conception

1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la gestion des demandes, tickets et suivi du temps.

[À compléter avec les diagrammes de séquence]

2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des demandes, tickets, sous-tâches et sessions de travail.

[À compléter avec le schéma de base de données]

III Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 3, incluant les modules de gestion des demandes, tickets et dashboards.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 3, notamment le tableau Kanban et les dashboards.

[À compléter avec les captures d'écran]

IV Tests

1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 3.

[À compléter avec les résultats des tests]

2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

CHAPITRE 6

Chapitre 6



Sommaire

6

I	Analyse	113
II	Conception	114
III	Réalisation	115
IV	Tests	116

Introduction

Dans ce chapitre, nous abordons le quatrième et dernier sprint du projet **SolidMaint**, couvrant la période du 2 mai au 31 mai 2026, soit quatre semaines de développement consacrées aux fonctionnalités avancées de collaboration, de gestion documentaire et d'intelligence artificielle. Nous commencerons par la présentation du backlog du sprint regroupant trente-quatre user stories réparties en six epics, suivie de la phase d'analyse et de conception, puis de la réalisation des interfaces intégrant un système de messagerie en temps réel, un module de gestion documentaire avec versioning, un système de notifications multicanaux et un moteur d'intelligence artificielle dédié à la catégorisation automatique des demandes et à la recommandation de solutions, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 10 - Messagerie et collaboration			
US67	En tant qu'utilisateur, je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	Sidebar avec channels, indicateurs messages non lus, statut présence utilisateurs	2
US68	En tant qu'utilisateur, je souhaite envoyer des messages dans un channel afin de communiquer.	Éditeur riche, mentions @, pièces jointes, livraison via WebSocket	2
US69	En tant qu'utilisateur, je souhaite répondre à un message afin de créer un fil de discussion.	Thread dédié pour réponses, notifications uniquement aux participants	2
US70	En tant qu'utilisateur, je souhaite mentionner des collègues afin d'attirer leur attention.	Autocomplétion @nom, notification push, badge mention non lue	1
US71	En tant qu'utilisateur, je souhaite éditer mes messages afin de corriger des erreurs.	Édition possible dans les 15min, indicateur "(modifié)", historique modifications	1
US72	En tant qu'utilisateur, je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	Suppression avec confirmation, remplacement par "[Message supprimé]", conservation pour audit	1
US73	En tant qu'utilisateur, je souhaite épingler des messages importants afin de les retrouver facilement.	Épinglage (permission requise), limite 10 par channel, possibilité désépinglage	1
US74	En tant que chef de projet, je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	Liste membres en ligne, filtrage par projet/équipe, dernière activité	1

ID US	User Story	Description technique	Effort (j)
US75	En tant qu'utilisateur, je souhaite voir qui est en ligne afin de savoir qui est disponible.	Indicateur statut (EN_LIGNE, ABSENT, NE_PAS_DERANGER), modifiable	1
Epic 11 - Gestion des documents			
US76	En tant qu'utilisateur, je souhaite uploader des documents afin de centraliser les fichiers.	Upload drag & drop, association contrat/projet/ticket, métadonnées (titre, description, tags)	2
US77	En tant qu'utilisateur, je souhaite télécharger des documents afin de les consulter hors ligne.	Téléchargement individuel/multiple (ZIP), vérification permissions, enregistrement consultation	1
US78	En tant qu'utilisateur, je souhaite gérer les versions des documents afin d'assurer la traçabilité.	Upload nouvelle version, numérotation auto (v1, v2), commentaire obligatoire	2
US79	En tant qu'utilisateur, je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	Historique pour traçabilité (qui, quand, durée)	1
US80	En tant qu'utilisateur, je souhaite archiver des documents afin de nettoyer la vue active.	Archivage avec confirmation, conservation données, restauration possible	1
US81	En tant qu'utilisateur, je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Calcul hash à l'upload, alerte si doublon, options (remplacer/créer version/annuler)	2
US82	En tant que chef de projet, je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Génération PDF professionnel (logo, en-tête, description, actions, temps, signature électronique)	3
US83	En tant que client, je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Accès PV depuis ticket, historique consultable, notification email à génération	1

ID US	User Story	Description technique	Effort (j)
Epic 12 - Notifications et alertes			
US84	En tant qu'utilisateur, je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	Icône avec compteur, dropdown listant notifications, marquage lu/non lu, accès direct élément	2
US85	En tant qu'utilisateur, je souhaite recevoir des notifications par email afin d'être informé hors connexion.	Préférences fréquence (temps réel/digest), templates professionnels	2
US86	En tant que membre technique, je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	Configuration Slack par contrat/projet, webhook URL, événements sélectionnables	2
US87	En tant qu'utilisateur, je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	Demande permission, notifications événements critiques, clic ouvre application	2
US88	En tant qu'utilisateur, je souhaite consulter l'historique de mes notifications afin de retrouver une information.	Liste complète avec filtres (type/statut/période), recherche, pagination	1
Epic 13 - Rapports			
US93	En tant qu'administrateur, je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	Rapport (tickets traités, temps passé, respect SLA), export PDF/Excel, graphiques	3
US94	En tant qu'administrateur, je souhaite générer des rapports de conformité SLA afin de justifier les performances.	Rapport taux respect par contrat/période, détail par ticket, calcul pénalités, PDF professionnel	3
US95	En tant que chef de projet, je souhaite générer des rapports d'activité afin de présenter l'avancement.	Rapport par projet/équipe/période (tickets traités, temps passé), graphiques évolution	2
Epic 15 - Versioning avancé			

ID US	User Story	Description technique	Effort (j)
US98	En tant qu'utilisateur, je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	Upload avec commentaire obligatoire, numérotation auto (v1.0, v1.1, v2.0), conservation versions	2
US99	En tant qu'utilisateur, je souhaite comparer deux versions d'un document afin de voir les différences.	Sélection deux versions, affichage côte à côte, mise en évidence modifications	2
US100	En tant qu'administrateur, je souhaite voir qui a consulté un document afin de suivre sa diffusion.	Liste consultations (date/heure/utilisateur, durée), statistiques	1
Epic 16 - Assistance intelligente			
US101	En tant que client, je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Analyse titre/description, suggestion catégorie, acceptation/modification possible	3
US102	En tant qu'administrateur, je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Apprentissage des validations/rejets, réentraînement périodique, suivi métriques précision	3
US103	En tant que chef de projet, je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Analyse contenu et contexte client, suggestion priorité, ajustable par chef projet	2
US104	En tant que développeur, je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Identification tickets similaires, suggestion solutions précédentes, acceptation/rejet	3
US105	En tant que développeur, je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Système notation (utile/pas utile), collecte retours, ajustement modèle	2
US106	En tant qu'administrateur, je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	Analyse similarité à création, signalement doublons, liaison automatique demandes similaires	2

ID US	User Story	Description technique	Effort (j)
Total Sprint 4 :			64 j

I Analyse

1 Diagramme des cas d'utilisation du Sprint 4

Cette section présentera le diagramme de cas d'utilisation du quatrième sprint, couvrant la messagerie, gestion documentaire, notifications et IA.

[À compléter avec le diagramme de cas d'utilisation]

2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 4.

[À compléter avec les descriptions textuelles]

II Conception

1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la messagerie, gestion documentaire et système d'IA.

[À compléter avec les diagrammes de séquence]

2 Modèle de données

Cette section présentera le modèle de données relatif à la messagerie, documents, notifications et modèles d'IA.

[À compléter avec le schéma de base de données]

III Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 4, incluant les modules de messagerie temps réel, gestion documentaire et IA.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 4, notamment la messagerie et le système de notifications.

[À compléter avec les captures d'écran]

IV Tests

1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 4.

[À compléter avec les résultats des tests]

2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]